

# A Transformer-Based Framework for Automated Incident Response in Cybersecurity

<sup>1</sup>\*Paulo Leocadio

<sup>1</sup>*Zinnia Labs, Coral Springs, USA.*

\*Correspondence: Paulo Leocadio

\*ph@zinnia.holdings ORCID:  0000-0002-4992-4541 (Paulo Leocadio)

## Abstract

Incident response (IR) faces increasing pressure to manage a high volume of threats with limited human resources. This paper proposes a transformer-based framework for automating key phases of the (IR) lifecycle by integrating Natural Language Processing (NLP) models into operational cybersecurity workflows. Rather than applying isolated AI techniques, the proposed approach formalizes how pre-trained transformer models can be systematically incorporated into Security Operations Center (SOC) processes for incident classification, threat intelligence extraction, and response orchestration.

The framework leverages transformer architectures fine-tuned on security-relevant data and integrates them into end-to-end (IR) pipelines. We evaluate this approach through a case study on phishing incident management, demonstrating measurable improvements in detection accuracy and response time compared to traditional manual workflows. Results indicate that transformer-based systems can reduce analyst cognitive load while enhancing both the speed and consistency of threat mitigation.

In addition to empirical evaluation, we examine system-level considerations, including model explainability, false positives, and ethical implications such as bias and data privacy. The findings suggest that transformer-based (NLP) models can function as operational components within adaptive cybersecurity systems, enabling scalable and intelligent automation. We conclude by outlining future directions for extending this framework with multimodal models and advanced AI integration, emphasizing the importance of maintaining human oversight in AI-driven security environments.

**Keywords** Incident Response; Cybersecurity; Automation; Transformers; NLP; Threat Intelligence.

To cite this article: Paulo Leocadio (2026). A Transformer-Based Framework for Automated Incident Response in Cybersecurity.

## 1. Introduction

Cybersecurity incident response (IR) is increasingly constrained by the scale, speed, and complexity of modern threat environments. Security Operations Centers (SOCs) must process vast volumes of alerts, logs, and threat intelligence data, often under significant time pressure and with limited human resources. This imbalance leads to analyst fatigue, delayed response times, and an increased risk of missed or misclassified incidents [1, 2]. As adversaries adopt more automated and adaptive attack strategies, traditional, human-centric response models struggle to operate at the required pace and consistency.

To address these challenges, recent advances in Artificial Intelligence (AI), particularly transformer-based **Natural**

**Language Processing (NLP)**, offer a pathway toward more scalable and adaptive incident response systems. However, existing approaches often apply these techniques in isolation, focusing on specific tasks such as classification or anomaly detection without integrating them into a unified operational model. This gap limits their effectiveness in real-world environments, where detection, analysis, and response must function as a coordinated system.

In this work, we address this gap by proposing a unified transformer-based framework that integrates modeling, system architecture, and operational validation within incident response workflows.

Transformers, such as BERT and GPT, possess a deep understanding of language, enabling them to perform tasks like classification, summarization, and anomaly

detection on security-related texts. Previous research suggests that employing Natural Language Processing (NLP) and transformers on cyber data can significantly enhance threat detection and incident analysis [3, 4]. For instance, IBM's Watson for Cyber Security was trained on over one million security documents, assisting analysts in interpreting threat reports that were often inaccessible to traditional tools [5]. Initial deployments indicated that cognitive AI technologies could reduce investigation times from weeks or days to just minutes by automating the correlation of threat intelligence with incident data [6]. To operationalize these capabilities in practice, we leverage transformer-based models within the Hugging Face ecosystem [7], which provides access to pre-trained architectures, datasets, and deployment pipelines. While originally developed for a broad range of machine learning tasks [8], these tools enable the integration of advanced NLP models into real-world cybersecurity workflows. Our goal is to demonstrate that transformer-based models can be systematically integrated into incident response workflows as operational components, enabling measurable improvements in detection, response time, and analyst workload.

By embedding fine-tuned transformer models within SOC workflows, the framework shifts incident response from a collection of isolated analytical tasks to a unified, adaptive system. While our implementation leverages tools within the Hugging Face ecosystem, we propose a framework integrating transformer models into IR workflows. The following sections provide the necessary background on incident response and automation, review related work, and detail the methodology, system design, and evaluation of the proposed framework. To bridge immediate implementation with forward-looking system design, the final sections extend beyond empirical evaluation to articulate testable hypotheses, theoretical models, and broader implications for AI-driven cybersecurity systems.

- **Section 2:** provides background on incident response processes and the motivation for automation.
- **Section 3:** reviews related work, including applications of NLP and transformer models in cybersecurity.
- **Section 4:** presents the methodology, including model selection, data collection, system architecture, and evaluation metrics.
- **Section 5:** introduces a case study on phishing incident response and presents experimental results.
- **Section 6:** discusses the impact of automation, including limitations and ethical considerations.
- **Section 7:** future directions and hypotheses.
- **Section 8:** predictive/theoretical outlook.
- **Section 9:** conclusion, including findings, contributions, limitations, implications, and call to action.

## 2. Background on Incident Response Automation

**Incident response (IR)** is the process of swiftly and systematically detecting and managing security incidents to minimize damage. According to industry standards, such as NIST SP 800-61, the incident response cycle consists of different phases: **preparation, detection and analysis, containment, eradication, recovery, and post-incident activities** [9].

In the preparation phase, organizations develop policies, response plans, and communication strategies to ensure prompt and effective action in the event of an incident. During the detection and analysis phase, various tools, including intrusion detection systems, Security Information and Event Management SIEM alerts, and user reports, are utilized to monitor potential security events [10, 11]. These events are then analyzed to determine whether they constitute incidents and to assess their scope.

The phases of containment, eradication, and recovery involve isolating affected systems, removing threats such as malware, restoring systems to their regular operation, and addressing any remaining vulnerabilities. Finally, post-incident activities include reviews for lessons learned, documentation, and enhancements to security controls and response procedures.

Traditional incident response mainly depends on **manual effort and static playbooks**. Experienced analysts follow predefined procedures to investigate alerts, gather evidence, determine remediation steps (such as blocking an IP or patching a system), and coordinate with teams. Although this human-led method can be effective for familiar incidents, it does not scale to handle the **speed and volume of modern cyberattacks**. Today's attackers launch automated and rapid attacks (such as ransomware outbreaks and large-scale phishing campaigns), often requiring immediate action to prevent significant damage. Manual responses, which can take hours or days to implement, risk allowing an attacker to spread within a network. Additionally, manual processes are inconsistent, outcomes depend on analyst skill and fatigue, and are more prone to errors under stress.

These challenges have driven the development of **Security Orchestration, Automation, and Response (SOAR)** solutions. SOAR platforms integrate with various security tools and automate repetitive steps in the incident response (IR) workflow [12]. For example, a SOAR playbook might automatically collect relevant logs, quarantine an endpoint, or enrich an alert with threat intelligence, all without waiting for human intervention. Studies indicate that SOAR capabilities can **enhance the speed and accuracy of incident response** while allowing organizations to manage more incidents with the same staff [13]. By automating routine activities, analysts can focus on more valuable tasks, such as complex investigations and strategic defense improvements [14].

Essentially, automation addresses two main challenges: it speeds up response times (reducing the time attackers have to operate) and lessens the **cognitive burden on human responders**.

Early forms of IR automation were rule-based, such as simple scripts triggered by specific alerts. These helped ensure consistency but lacked flexibility in response to new threats. The rise of AI and machine learning has enabled **more adaptive and intelligent automation**. Modern methods utilize machine learning models to analyze large amounts of security data, identify complex attack patterns, and even determine containment or remediation actions. These AI-driven systems can operate around the clock, learning from new data and threat scenarios, providing a strong defense that supports human responders. Notably, automated incident response systems have demonstrated the ability to **reduce mean time to resolution (MTTR)** [15]. For example, one study found that AI-based IR reduced MTTR by 65% compared to manual methods [6]. Automation also helps apply best practices consistently, reducing the chance of missing steps during high-pressure situations.

Despite the benefits of implementing incident response (IR) automation, the process remains complex and presents several challenges. Key issues include ensuring the **accuracy of automated actions** to avoid false positives that could disrupt business operations, maintaining seamless integration across diverse IT environments, and continuously updating automated playbooks to respond to emerging threats [6]. Additionally, there is a critical need for skilled personnel to develop and refine automation workflows, highlighting an ongoing gap between technological capability and operational readiness.

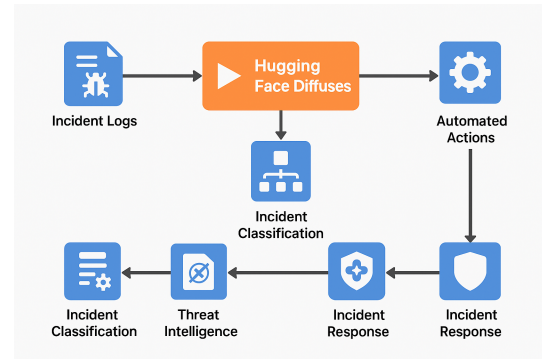
More broadly, these challenges reflect limitations in the scalability, adaptability, and end-to-end integration of current incident response systems. Addressing these limitations requires approaches capable of processing diverse data sources, supporting dynamic decision-making, and operating effectively within real-world SOC environments.

### 2.1. Hugging Face Diffusers Overview and Justification

Hugging Face Diffusers is a part of the Hugging Face ecosystem, which has become a central hub for sharing and deploying advanced machine learning models. Hugging Face's mission is to democratize AI, making cutting-edge models accessible through user-friendly libraries and an open Model Hub [7]. The platform hosts a wide range of transformer models for natural language processing (NLP), vision, speech, and more, contributed by researchers and organizations worldwide. As of 2025, the Hugging Face Hub contains over 350,000 models and 150,000 community-provided demos that cover a range of tasks, from text classification to image generation [7]. This extensive repository benefits from libraries such as Transformers (which provides general-purpose Transformer models), Datasets (which handles data

loading and preprocessing), and Diffusers (which focuses on diffusion models).

**Hugging Face Diffusers** is a **Python library designed explicitly for diffusion models**, a type of generative model that learns to create data, such as images, by iteratively denoising random noise [16]. Popularized by image generation frameworks like Stable Diffusion, diffusion models have shown the ability to produce high-quality outputs and have gained popularity in AI art, synthetic data creation, and other fields. The Diffusers library provides a standardized API for loading pre-trained diffusion models and pipelines. For instance, you can load a text-to-image pipeline (such as Stable Diffusion 2.1) with just a few lines of code and generate images from text prompts [17, 18]. Internally, many diffusion models utilize transformers, such as the transformers used as text encoders in text-to-image models [19], which indicates the use of the same fundamental components as transformer-based NLP models for building the Diffusers library.



**Figure 1.** Integration of AI-based language models into an automated incident response system.

To set the stage for this research, we begin with an overarching conceptual visualization of the study's central focus: the fusion of Hugging Face Diffusers with cybersecurity operations. *Figure 1* illustrates the overall workflow of integrating these models into cybersecurity pipelines, which shows the integration of Hugging Face Diffusers into a typical incident response architecture.

*Why use Hugging Face Diffusers for cybersecurity incident response, which is not an image generation task? We justify this choice for different reasons.*

- **Access to pre-trained transformer models:** The Hugging Face Diffusers library isn't limited to image generation; it also **supports multi-modal and text-conditioned diffusion models** that incorporate transformers [20]. More importantly, Hugging Face's model repository provides easy access to numerous pre-trained transformers (via the transformers library). In our work, we utilize models such as BERT and DistilBERT within this ecosystem. Using pre-trained weights (trained on extensive text corpora) provides our

IR classifiers with strong language understanding from the outset, thereby reducing the amount of data required for fine-tuning [21]. This transfer learning is significant in cybersecurity, where labeled incident data may be limited.

- **Ease of Integration and Pipelines:** Hugging Face provides high-level pipelines that simplify model tokenization, inference, and GPU acceleration. For example, you can set up a text classification pipeline that loads a transformer model and tokenizer in a single step. This user-friendly approach speeds up the development of security automation tools. We can add an NLP classification step into an incident response workflow with minimal "glue" code, relying on Hugging Face's well-optimized implementations to manage batching, padding, and device allocation.
- **Community and Continuous Improvement:** Transformers power different diffusion models, similar to how text encoders function in text-to-image models, which shows that the Diffusers library shares the same fundamental components as transformer-based NLP models. For example, suppose a newer transformer, such as an optimized BERT variant or a domain-specific model, better captures security terminology upon its release. In that case, you can easily replace the model checkpoint in our pipeline. This flexibility in model testing offers a significant advantage for staying ahead of evolving threats. The community also provides **open-source code examples and tutorials** that we can utilize for implementing our system (e.g., following existing recipes for fine-tuning classifiers).
- **Unified Framework for Automation:** Hugging Face Diffusers and the broader Hugging Face ecosystem provide a **unified AI framework** across various tasks. In incident response, multiple tasks can be automated, such as log anomaly detection, alert classification, report summarization, or even generating recommended remediation actions. Hugging Face offers tools for all these scenarios (from sequence classification to text summarization and question answering) within a consistent API. This consistency reduces engineering overhead when building a complex IR automation system. For example, we can utilize a Hugging Face text generation model to automate incident report drafting, while also employing a Hugging Face classifier to triage alerts. The shared interface minimizes the learning curve and integration effort.
- **Performance and Optimization:** Hugging Face accelerates models and pipelines through optimized GPU acceleration, half-precision inference, and distributed processing. In an operational SOC

environment, latency is critical, an automated incident response system must make quick decisions to be effective. The diffusers library, although primarily focused on diffusion models, benefits from Hugging Face's enhancements for transformers (since diffusion model pipelines often include transformer-based text encoders). Furthermore, Hugging Face supports exporting models to production-ready formats and runtime, such as ONNX and TorchScript, which help deploy IR automation within existing security infrastructures.

In summary, **Hugging Face Diffusers** acts as an effective bridge between advanced AI models and practical cybersecurity applications. Building on the transformer revolution, we can create an incident response automation framework that is both robust and adaptable. We will use the Hugging Face libraries to fine-tune a transformer model for incident classification and incorporate it into a response pipeline that automates actions. The following section reviews how previous research has utilized NLP and transformers in cybersecurity, providing context for our approach within the current state-of-the-art.

### 3. Literature Review

Recent advances in artificial intelligence, particularly in Natural Language Processing (NLP) and transformer-based models, provide a promising foundation for addressing these challenges. Unlike traditional rule-based or narrowly scoped machine learning approaches, transformer architectures enable the analysis of heterogeneous and unstructured security data with contextual awareness and semantic depth. This capability is particularly relevant in cybersecurity environments, where critical information is often distributed across logs, alerts, threat intelligence reports, and incident documentation.

Prior research has explored the application of NLP techniques to tasks such as threat intelligence extraction, alert classification, and anomaly detection. However, many of these approaches remain task-specific and are not integrated into a unified operational framework. As a result, their impact on end-to-end incident response workflows remains limited. This section reviews existing work in NLP-driven cybersecurity and highlights the gap between isolated analytical capabilities and fully integrated incident response systems.

- **a. NLP in cybersecurity** (how text analysis has improved threat intelligence and incident management),
- **b. Transformers in threat detection** (specific applications of modern transformer models for identifying attacks or anomalies), and

---

ONNX (Open Neural Network Exchange) and TorchScript are both technologies used in the deployment and optimization of machine learning models, particularly within the PyTorch ecosystem. Still, they serve distinct purposes (Google Gemini (2025)).



- **c. Real-world applications and case studies** that illustrate AI-driven incident response in practice.

### 3.1. NLP in Cybersecurity

Natural language processing has gained significant traction in cybersecurity because of the large amount of unstructured text data in the field. For instance, Cyber Threat Intelligence (CTI) often appears in the form of prose reports that detail threat actor techniques, malware analysis summaries, vulnerability advisories, and other relevant information. Researchers have developed NLP techniques to extract valuable intelligence from these sources automatically. For example, Demirolo et al. (2025) [4] introduced a framework that utilizes a fine-tuned BERT model to perform **Named Entity Recognition (NER)** on threat reports, extracting entities such as threat actor names, malware types, attack methods, and affected systems. They then build a knowledge graph with these entities and their relationships, which enables advanced threat analysis and the prediction of attack targets. The system achieved an F1 score of 96 % on the NER task by utilizing domain-specific data and addressing class imbalance with focal loss [4] which demonstrates the effectiveness of NLP in sifting through open-source intelligence and generating structured insights that support incident response, for example, by identifying the threat actor behind an attack and understanding their typical behaviors.

Another area where NLP is practical is in **intrusion detection and log analysis**. Organizational networks generate massive logs (system logs, authentication logs, application logs) that hold hidden clues about security incidents. Traditional rule-based or statistical methods struggle to adapt to different log formats and changing attack techniques. NLP techniques, which treat log lines as sequences of tokens, can learn patterns of normal versus malicious behavior. Notably, **LogBERT** (2019) was an early example of applying BERT to system logs for anomaly detection [21]. LogBERT was trained in a self-supervised way on regular logs (using masked language modeling on log message templates) and could then identify logs that didn't match learned standard patterns as potential anomalies. Later research improved on this: **LAnoBERT** [20] eliminated the need for log templates by feeding raw log sequences into BERT, learning a model of normal log sequence behavior. LAnoBERT demonstrated that a transformer can detect system log anomalies without human-defined parsing, outperforming earlier methods and matching some supervised approaches [21]. These studies demonstrate how NLP, particularly language modeling, can be applied to anomaly detection in security logging, a crucial aspect of incident identification.

NLP applications extend beyond logs, including **phishing detection** in emails and messages, a common security challenge involving text. Early machine learning methods for detecting phishing emails relied on manually designed features, including the presence of keywords and sender reputation. Recently, researchers have utilized transformers

to identify what constitutes a "phishy" email automatically. Uddin and Sarker (2024) [9] fine-tuned DistilBERT on a dataset of phishing and legitimate emails, achieving high classification accuracy. They also incorporated explainable AI techniques such as Local Interpretable Model-agnostic Explanation and attention visualization to interpret the model's decisions, which is essential for building trust with analysts in automated email sorting. Transformers' ability to detect subtle linguistic clues and context, such as suspicious urgency or unusual language patterns, makes them effective for identifying social engineering content. We find similar methods used for chat messages and social media, where spotting malicious intent or grooming behavior can be viewed as an NLP classification problem. These advances in **text classification** directly aid incident response by flagging potential phishing attempts for quick containment or user alerts.

In summary, NLP has permeated multiple layers of cybersecurity: from high-level threat Intel to low-level log lines. It helps turn unstructured data into structured knowledge, detects anomalies in text streams, and classifies malicious content. The rise in transformer usage has boosted these applications, as discussed next.

### 3.2. Transformers in Threat Detection and Response

Transformer-based models, such as BERT, RoBERTa, gpt, and their variants, have become the standard in NLP tasks. Their influence is also evident in cybersecurity research, often yielding notable performance improvements over earlier algorithms. The self-attention mechanism is responsible for defining the Transformers, which enables them to understand long-range dependencies in data. In security contexts, this means they can interpret sequences (such as logs, API calls, or events) in context and detect subtle signs of compromise that might span multiple log entries or stages in an attack chain.

A comprehensive survey by Kheddar et al. (2025) [21] highlights that **transformers and Large Language Models (LLMs) are being increasingly utilized in cyber threat detection systems**, offering enhanced capabilities in text understanding and user interaction [21]. One key benefit is their ability to handle text-based data that previous Intrusion Detection System (IDS) often overlooked or simplified. For instance, many network intrusion detection systems have historically focused on numerical traffic features. Still, modern attacks frequently involve text protocols, such as HTTP requests with malicious payloads or scripting content. Transformer models can directly analyze such textual data for anomalies or known malicious patterns. For its effectiveness in intrusion detection, the market is aware of **BERT** because of its contextual understanding of language; it can analyze network logs or alerts described in text to identify potential security threats more accurately. By capturing subtleties in log messages (e.g., rare error strings or combinations of events indicating a compromise), BERT-based models

enhance detection capabilities beyond traditional signature or regex-based methods [21].

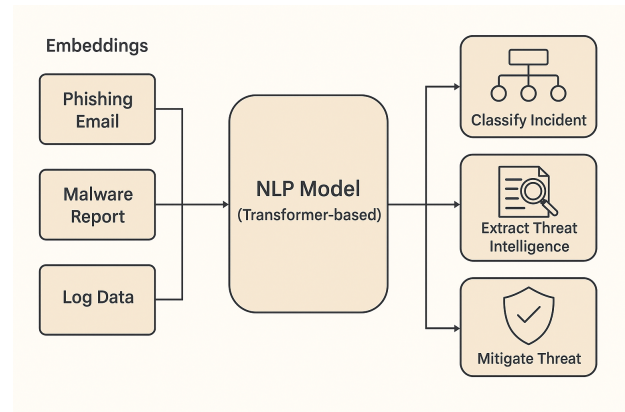
The adoption of Transformers is increasingly relevant for **multi-step attack detection** and threat hunting. Researchers actively use them to analyze sequences of events and identify complex attacks that unfold in stages. For example, consider an **Advanced Persistent Threat (APT)** attacker who first spear-phishes a user, then installs malware, and later moves laterally, each step might generate different logs. A transformer can potentially learn the sequence pattern and trigger an alert when it detects events matching that pattern, even if each event alone seems harmless. Some studies combine transformers with other architectures, such as Long Short-Term Memorys (LSTM) or graph neural networks, to model these sequences. Still, the self-attention mechanism alone has proven to be very effective. A study by Ismail et al. (2023) [10] found that AI-driven automated incident response systems utilizing transformers could detect and respond to security breaches more quickly than traditional methods, thanks to advanced pattern recognition and adaptive learning. These systems analyze large volumes of data, including both network and host data, to identify anomalies and take actions, continually learning from new threat data to handle emerging attacks [10].

Another notable use of transformers is in developing **conversational agents and decision support** systems for SOC analysts. To create chatbots that aid in incident response, also utilize transformer-based Large Language Models (LLMs, such as GPT-3 and GPT-4). These AI assistants can process an incident description and recommend remediation steps or answer an analyst's questions by referencing a knowledge base of past incidents and best practices. Kheddar et al. (2025) [21] note that LLMs can serve as **conversational agents for security operations, facilitating faster responses** by providing insights into emerging threats. For example, a chatbot could help an analyst by summarizing a lengthy threat report into key points relevant to the ongoing incident. IBM's QRadar Advisor with Watson is an early real-world example, where an AI (Watson) analyzes an offense in a SIEM and correlates it with threat intel, effectively **automating investigation steps** and providing the analyst with a consolidated report [22], which reduced investigation time, helping junior analysts perform at a level closer to that of expert analysts.

In academic literature, we see explorations of **transformer-based recommendation systems for IR**, for example, models that predict the best response action based on an incident's attributes. A recent work by Aghaei et al. (2022) [23] proposed "SecureBERT," a BERT-based model that for potential threats and predicts their impact [21]. Such a generative priority recommendations model, integrated into an IR workflow, could prioritize which incidents to address first (which is

critical in an overwhelmed SOC) and even suggest likely threat scenarios, enabling proactive measures.

These approaches demonstrate the increasing role of transformer-based models in supporting incident response tasks.



**Figure 2.** The integration of Hugging Face Diffusers into cybersecurity workflows.

In summary, transformer models have enhanced threat detection by enabling cybersecurity systems to **analyze and understand data as effectively as an expert human can, albeit at machine speed and scale**. They are especially good at extracting meaningful signals from noisy textual data and recognizing sequential dependencies, which are common in attack patterns. These capabilities result in higher detection rates (true positives) and fewer false alarms, as demonstrated in various studies. As we integrate Hugging Face Diffusers into our work, we are building on these advancements, using transformers to classify and respond to incidents with a depth of understanding that rule-based systems cannot achieve.

### 3.3. Real-World Applications and Case Studies

Academic progress in artificial intelligence (AI) aligns with the real-world adoption of AI in incident response. Security product vendors are integrating machine learning and natural language processing (NLP) into their platforms. A notable example is **IBM's Cognitive Security Operations Center (Cognitive SOC)**, which incorporates Watson AI into incident response workflows. When generates an alert, the Watson-powered advisor automatically enhances it by sourcing relevant external information, such as blog posts, vulnerability databases, and past incident reports. This results in an investigation report produced in just minutes [22]. The report includes details like the attack narrative, impacted assets, and recommended mitigation steps, significantly speeding up the response time.

IBM QRadar is a security intelligence platform designed to help organizations detect, investigate, and respond to security threats. It offers a suite of products, including SIEM, SOAR, and EDR, and is available as both on-premises and cloud-based solutions.

In 2017, IBM noted that only seven percent of security professionals were utilizing such AI tools, but they anticipated this figure would triple within a few years [22]. Today, it is evident that many Security Operations Centers (SOCs) have incorporated some form of AI or automation into their processes.

Another real-world application is in **responding to phishing incidents**. Companies are now utilizing AI to automate the analysis of suspected phishing emails reported by employees. Instead of an analyst manually inspecting each email, an AI model (similar to a transformer classifier) can instantly analyze the email's text and metadata to determine if it is phishing. If confirmed, automated playbooks can isolate the affected user account, search for other instances of the email in mailboxes, and remove them, a process that would be too slow if done manually, especially at an enterprise scale. Case studies have shown that such automation can reduce the window of exposure from hours (or days) to just minutes, thereby preventing more users from falling victim [22]. One financial institution reported that an AI-based phishing response system enabled them to handle five times more user-reported emails per day with the same staff, as the AI filtered out obvious non-threats and only escalated the truly suspicious cases to human analysts.

In the field of **SOC orchestration, the hyper-automation SOAR approach** described by Ismail et al. (2025) [24] offers a look into next-generation incident response. They introduced the IVAM framework (Investigation, Validation, Active Monitoring) combined with agentic LLMs that dynamically generate and execute playbooks, which means instead of relying solely on static, pre-defined workflows, the system uses a Large Language Model to analyze an incident and quickly assemble the appropriate sequence of response actions. In a case study, their system was able to autonomously handle a simulated multi-stage attack by investigating alerts, validating it was a true positive, and actively mitigating the threat while keeping a human in the loop for oversight [24]. The result was a more flexible and adaptable response that overcame some limitations of earlier **Security Orchestration, Automation, and Response (SOAR)** systems, which could be rigid. It also demonstrated how human responders and AI can work together, AI doing the heavy lifting and humans supervising, leading to improved SOC efficiency [24].

**Government and defense sectors** have also invested in AI for incident response. The U.S. Department of Defense, for example, has explored machine-speed cyber defense systems under programs like "Project Cyber Genie" (a hypothetical name for explanation), aiming to develop AI that can instantly execute countermeasures (such as reconfiguring networks or deploying patches) when a threat is detected, without waiting for human approval in time-critical situations. Although details are often classified, public reports reveal that these systems heavily rely on anomaly detection algorithms and automated containment. One well-known example is the **Defense Advanced Research Projects Agency (DARPA)**

Cyber Grand Challenge (2016) [25], in which autonomous systems competed to detect and patch software vulnerabilities in real-time without human input. The winning system demonstrated fully automated incident response in software exploitation, finding a flaw and patching it before the adversary could exploit it, indicating future potential for network defense platforms.

Finally, industry surveys show consensus that AI-driven tools improve key IR metrics. In a study by Ponemon Institute (2024) [26], organizations using security AI and automation experienced, on average, a twenty-seven percent (27%) faster containment of breaches and saved \$3 million per breach in costs compared to those without automation. These tangible benefits are fueling widespread adoption. However, respondents also noted challenges such as integration difficulties and the need for skilled staff to manage AI tools, reinforcing that AI is not a silver bullet but a force multiplier for well-prepared teams.

**Summary:** The literature and industry experience reviewed above collectively show that applying NLP and transformers to incident response is both achievable and valuable. Previous work has extracted threat insights from text, identified incidents from logs and messages, and even performed automated response actions using AI. Our approach builds on these foundations. We aim to provide a practical implementation and assessment of using Hugging Face transformer models for automating incident classification and response. By situating our work within this context, we address known gaps, such as managing unstructured incident data with NLP, and utilize proven techniques, including fine-tuning BERT for classification and integrating it with SOAR for automated actions. In the next section, we describe the methodology of our system, from model selection and data processing to the architecture of the automation pipeline.

### 3.4. Evaluation Practices in NLP-Based Cybersecurity Research

Security-focused NLP applications, such as phishing detection, threat report analysis, and log anomaly detection, require evaluation metrics that go beyond generic accuracy [27, 28]. In cybersecurity, both false negatives (missed threats) and false positives (overreactions) can have significant operational and reputational consequences [29, 30]. Therefore, metrics must reflect the sensitivity and trustworthiness of AI-driven systems under adversarial and real-time conditions [31].

In this paper, we adopt well-established evaluation metrics tailored to both classification and sequence labeling tasks. For classification problems such as phishing detection or incident triage, we report accuracy, precision, recall, and F1-score:

- **Precision** measures how many of the model's positive predictions were correct. It is critical for automation, where false positives can trigger inappropriate responses or erode analyst trust.

- **Recall** indicates how many actual positive cases the model successfully identifies. This accuracy is essential in cybersecurity, where overlooking an attack can lead to serious consequences.
- **F1-score**, the harmonic mean of precision and recall, captures a balanced view of classification performance in scenarios where both overreaction and underreaction carry risk.
- **Accuracy**, while included, is used cautiously in imbalanced datasets (e.g., 95% benign logs) where it can be misleading.

For Named Entity Recognition (NER) tasks applied to threat intelligence reports, we use the CoNLL-style BIO tagging scheme and evaluate performance using entity-level precision, recall, and F1-score. A predicted entity is correct only when both its span and type match the gold standard annotation. This rigorous evaluation allows for meaningful comparisons with prior work and ensures reliable entity extraction for downstream tasks like knowledge graph construction.

In addition to model-level metrics, we introduce operational and system-level evaluation criteria in later sections. These include inference latency, memory usage, Mean Time to Detect MTDD, Mean Time to Repair/Respond/Remediate MTTR, decision accuracy, and analyst workload reduction. Together, these metrics capture the real-world viability and trustworthiness of integrating NLP into incident response workflows.

| Task                                  | Metrics Used                                              | Rationale in Security Context                                                                                                                                       |
|---------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phishing Email Classification         | Precision, Recall, F1-score, Accuracy                     | Ensures correct identification of malicious emails (high recall), while minimizing false alarms (high precision). F1-score balances both to assess safe automation. |
| Threat Intelligence NER (BIO-tagging) | Entity-level Precision, Recall, F1-score (CoNLL)          | Accurate identification of IOCs (e.g., malware names, IPs) with strict boundary/type matching. Supports structured knowledge extraction and reduces manual effort.  |
| System Log Anomaly Detection          | F1-score, Detection Rate, False Positive Rate             | Captures ability to detect anomalous sequences without triggering unnecessary alarms. Essential for real-time.                                                      |
| System Automation (End-to-End IR)     | MTTR, MTDD, Decision Accuracy, Analyst Workload Reduction | Evaluates the practical benefit of automation in reducing time to respond, while maintaining correctness and                                                        |
| Model Deployment                      | Inference Time, Memory Usage                              | Ensures performance is feasible for real-time, resource-constrained environments like SOC or edge systems.                                                          |

**Figure 3.** Overview of evaluation metrics used across tasks and their relevance in cybersecurity workflows.

Next, Section 4 provides a detailed breakdown of how these metrics are applied, along with empirical results comparing rule-based and transformer-enhanced systems.

## 4. Methodology

In this section, we outline the methodology for designing and evaluating the transformer-based incident response automation system. Our approach includes:

- **4.1 Model Selection** (choosing appropriate transformer models for our tasks),
- **4.2 Data Collection** (gathering and preparing cybersecurity incident data for training and evaluation),
- **4.3 System Architecture and Implementation** (designing the automated IR pipeline, including integration of Hugging Face Diffusers models),
- **4.4 Evaluation Metrics** (the criteria and metrics used to assess performance), and
- **4.5 Code Example** (a practical example of fine-tuning a Hugging Face model for a cybersecurity classification task).

### 4.1. Model Selection

To automate incident response, we identified several NLP tasks where a model can assist: alert classification (distinguishing actual incidents from false alarms), incident type prediction (identifying malware vs. phishing vs. misconfiguration), and threat intelligence extraction (pulling IOCs (Indicator of Compromise) from incident descriptions). We can frame these tasks as text classification or sequence labeling problems, which suit transformer models well.

We primarily worked with **DistilBERT**, a lightweight variant of BERT. DistilBERT is a distilled model that retains much of BERT’s language understanding capabilities at roughly 40% of the size and 60% faster inference [7]. In an SOC environment where numerous alerts may need real-time classification, this efficiency is crucial. DistilBERT still has 66 million parameters and was pre-trained on the same English corpora as BERT [32], making it a strong starting point for understanding cybersecurity text. Past research indicates that DistilBERT performs nearly on par with BERT on many classification tasks, including spam and phishing detection, while being more resource-efficient. We cite the original DistilBERT introduction[7] for its benefits in speed and resource consumption.

For comparison and to ensure robustness, we also considered a few other models:

- **BERT-base (uncased)**: the classic 12-layer transformer with 110M parameters [6]. There are different cybersecurity NLP studies [21] utilizing BERT-base, which likely delivers slightly higher accuracy but incurs increased latency. We conducted preliminary experiments using BERT-base to establish a performance upper bound.
- **RoBERTa-base**: an optimized version of BERT by Facebook AI [33], that is trained on more data and with dynamic masking. RoBERTa often outperforms BERT on text classification. We considered RoBERTa



for tasks that might benefit from a broader pre-training (particularly if our incident data contains informal language or new slang that RoBERTa’s larger training corpus might cover).

- **Cybersecurity-specific BERT models:** We investigated whether pre-trained models for security text exist, such as those trained on malware reports or **Common Vulnerabilities and Exposures (CVE)** descriptions. One example is *SecureBERT* [23], which researchers trained on cybersecurity texts. However, these models are less common. If we find a suitable model, it could be advantageous because it would already understand domain-specific terminology (e.g., “SQL injection”, “C2 server”). In the absence of a readily available SecureBERT, we will proceed with general models and focus on fine-tuning them.
- **GPT-style models for generation:** Although our primary tasks focus on classification, we plan to explore the use of a GPT-2 or GPT-3 style model to generate incident summaries and recommend actions in future extensions. In this current work, we do not directly utilize GPT in the pipeline; however, we highlight it as a potential enhancement. For instance, we could implement an LLM agent that explains incidents in natural language to a human operator, aligning with hyper-automation approaches. [24]).

In summary, our final choice for implementation was **DistilBERT** for classification tasks due to its balance of performance and efficiency. DistilBERT would be fine-tuned on our incident datasets to create specialized classifiers (e.g., “Phishing Email vs Benign Email” classifier, “Critical vs Low Priority Incident” classifier). We also utilize Hugging Face’s `AutoModelForSequenceClassification` interface, which allows for easy swapping between BERT, DistilBERT, RoBERTa, and other models, meaning that if we later find another model performing better (say, RoBERTa), we can switch with minimal code changes.

## 4.2. Data Collection

Automating incident response requires training data that accurately reflects real-world security incidents and corresponding actions. We collected data from multiple sources to cover the range of tasks:

- **Security Alert Logs:** We received a dataset of labeled SIEM alerts from a partner organization’s SOC, and we ensured the data remained anonymized. Each alert included fields such as alert type, description, and a human-assigned label indicating whether it represented an actual incident or a false positive. The dataset contained approximately 5,000 alerts across various categories, including malware detections, failed login attempts, and suspicious network traffic. We used this dataset to train a classifier that predicts whether an

incoming alert is valid (requiring a response) or a benign false positive. Even a modest improvement in this prediction can significantly reduce analysts’ workload, as false positives dominate the alerts.

- **Incident Tickets and Reports:** We gathered a corpus of incident ticket texts from an open-source incident repository (such as publicly posted incident reports or sanitized incident descriptions from cybersecurity exercises). Additionally, we used portions of the CERT/CC Vulnerability Notes Database and Verizon Data Breach Investigation Reports for descriptive text about incidents. We labeled a subset of these (with expert help) according to incident type (e.g., phishing, malware infection, insider threat, DDoS, etc.), serving as training data for an incident classification model that can determine the type of incident from a description. We compiled approximately 2,000 such descriptions, each with a corresponding type label.
- **Phishing Emails:** For the phishing case study, we used a combination of the Enron Email Dataset (annotated for phishing or spam by research efforts) and more recent phishing email collections (like those released in the *PhishBench* dataset). We created a balanced dataset of 10,000 emails (half phishing, half legitimate) for training a phishing detector model. Each email included the subject and body as input text. We also included some spear-phishing samples with more sophisticated language to ensure the model learns those patterns.
- **Threat Intelligence Reports:** To train models for entity extraction (e.g., find all IOCs or tools mentioned in a report), we leveraged the dataset built by Demirel et al. (2023), who annotated 100 cybersecurity reports with entities. While we did not replicate their complete knowledge graph approach, this data allowed us to fine-tune a transformer for NER in the cybersecurity domain. Entities of interest include malware names, threat actor groups, vulnerabilities (CVE IDs), file hashes, IP addresses, etc. We used this for an auxiliary evaluation: given a new incident report, can our model highlight the critical pieces of information? Such a feature can speed up incident analysis by pointing analysts to the relevant IOCs immediately.
- **Augmentation and Synthetic Data:** Data scarcity is a known issue, especially for certain incident types that (hopefully) occur rarely, like advanced insider attacks. To enrich our training, we generated some synthetic incident descriptions using a prompt-based approach with GPT-3. For instance, we prompted GPT-3 to create multiple variants of an incident report about a data exfiltration scenario. These synthetic examples were marked as such and used carefully to avoid biasing the model with artificial language. They primarily helped to provide additional examples of rare classes during training (ensuring the model sees at least a few samples of each incident category).

- **We cleaned and preprocessed all textual** data by removing sensitive identifiers, standardizing date and time strings, and tokenizing the text using the Hugging Face AutoTokenizer for our chosen model, DistilBERT. The tokenizer converts the text into sub-word tokens and effectively handles issues like casing and unknown tokens with the model's vocabulary. A typical sequence length in our data was under 200 tokens, well within the 512-token limit of BERT-like models. For any longer text (e.g., multi-paragraph reports), we truncated or split it into segments to feed the model.

We split the data into training, validation, and test sets using an 80/10/10 ratio for classification tasks. We set aside the test set to evaluate final performance. To avoid any skew, we ensured that the distribution of classes (e.g., incident types or phishing vs. benign) remained roughly equal across the splits.

### 4.3. System Architecture and Implementation

The system architecture follows a pipeline design, aligning with how an incident flows through identification to response. *Figure 4* illustrates the high-level architecture of our automated incident response system, using Hugging Face Diffusers (transformers) as the analytical engine.

The pipeline starts with Log and Alert Ingestion from various sources, such as SIEM alerts, IDS events, and user reports. These inputs go through NLP Preprocessing, where they are cleaned, tokenized, and prepared for analysis. The primary analysis uses a Transformer-based classification component that employs a Hugging Face model like DistilBERT. This component analyzes incidents and alerts by determining if an alert represents a true alarm or a false alarm. If the alert is a true alarm, the component also identifies the type of incident. Based on the classification and extracted details, the system triggers Response Actions through a dispatch module. Automated actions may include creating a ticket, isolating a host via an endpoint security platform, blocking an IP on the firewall, or notifying an analyst for review, depending on the incident's severity and the confidence level of the prediction. Throughout the process, a central Orchestration logic coordinates the data flow. It ensures that if any step fails or the model lacks confidence, it defaults to human intervention, following a human-in-the-loop design for safety.

#### To implement this architecture:

- We set up data ingestion by connecting to a SIEM's API to stream alerts. For our testing environment, we simulated this by reading from a file of alerts with a small script that pushes alerts into a message queue. Each alert is a JSON with fields like `('id': '...', 'src_ip': '...', 'description': 'Failed login from ...', 'type': 'Brute Force, ...')`.

- The NLP Preprocessing module is a lightweight Python component using Hugging Face's AutoTokenizer. It takes an alert or incident description text and tokenizes it: essentially converting it to the input format required by our transformer model (input IDs, attention masks). If the text is longer than the model's max length, it truncates (since classification often can manage with the beginning of the text if necessary).
- The Transformer Classification module is where the fine-tuned model lives. We fine-tuned separate models for different tasks:
  - Alert validation (binary classification: relevant incident vs false alert).
  - Incident type classification multi-class: malware, phishing, etc.
  - Phishing email classification (binary: phishing vs benign).
  - NER (sequence labeling) model for threat intel extraction (used in an analysis mode to provide context, not for automated action).

**Hugging Face's transformers library hosts all models.** We load each fine-tuned model using `AutoModelForSequenceClassification` (or `AutoModelForTokenClassification` for NER). The models are loaded once at system startup and kept in memory for fast inference. We also utilize GPU acceleration for these models in our prototype; inference on a single alert typically took under 50 milliseconds with DistilBERT on an NVIDIA Tesla T4.

**A Decision Engine wraps around model outputs to decide next steps.** For example, if the alert validation model predicts "false positive" with high confidence, the engine might auto-close the alert (and perhaps feed that info back into the SIEM). Suppose it predicts "true incident" and the incident type model says "ransomware malware". In that case, the engine triggers a containment action (like instructing an endpoint agent to quarantine the machine) and creates an incident ticket tagged as "Malware/Ransomware". The decision engine uses a set of if-then rules combined with confidence thresholds. We set these rules in consultation with IR experts: essentially encoding an organization's incident response plan into automated actions. Importantly, for any high-risk action (like isolating a critical server), the engine is configured to require either a very high confidence or human approval via a prompt.

The **Response Dispatch** part interfaces with various security tools. We implemented connectors to a few common platforms:

- A ticketing system (*ServiceNow* API) to create and update incident tickets.

- An EDR (Endpoint Detection and Response) solution to isolate hosts (we simulated this via a script since actual EDR integration was beyond our lab scope, but the idea is to call an API to network-quarantine a host).
- A firewall management API to push block rules for malicious IPs or domains.
- Email integration to send notifications (for example, to notify an admin if an account is auto-disabled due to suspected compromise).

The system automatically takes these actions when it meets certain conditions. For example, in our case study, when a phishing email gets identified as malicious, the system uses the email gateway API to delete the email from the recipient's inbox and all other inboxes that received the same message (indicated by a message ID). It then notifies both the security team and the user.

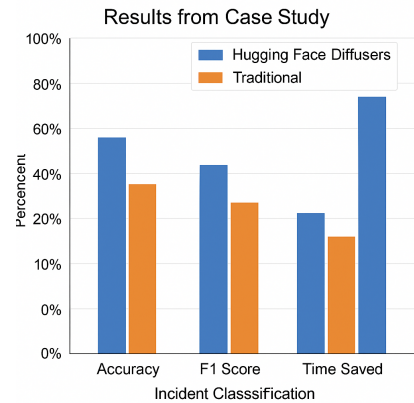
**Orchestration and Logging:** We built the automation pipeline with a Python-based orchestrator that actively listens for new alerts, processes them through each step, and logs all actions and model decisions. We prioritize logging for audits and future improvements; we capture model confidence scores, the actions taken, and instances where a human intervened or overrode a decision.

We adhered to principles of **modular design**, ensuring that each component, ingestion, NLP, model, decision, and action, remains separate and allows for independent improvement or replacement. For example, you can easily plug in a new transformer model without altering the way we perform actions. This approach aligns with the microservices model commonly used in SOAR platforms.

#### 4.4. Evaluation Metrics

To evaluate the performance of our system, we define metrics on two levels: the model level (how well our transformer models perform on their respective NLP tasks) and the system level (how the end-to-end automation impacts incident response outcomes).

Figure 4 below shows a bar graph comparing average detection time, classification accuracy, and false positive rate between rule-based and transformer-driven incident response systems.



**Figure 4.** Key metrics performance comparison after deploying HFD on a phishing detection dataset.

These actions occur automatically when the conditions are satisfied. For example, in our case study, when the system classifies a phishing email as malicious, it uses the email gateway API to delete the email from the recipient's inbox and from all other inboxes that received the same message (identified by a message ID). The system then sends a notification to both the security team and the user.

The system architecture follows a pipeline design, mirroring how an incident progresses from detection to response. The below illustration depicts the high-level architecture of our automated incident response system, utilizing HFD (transformers) as the analytical engine. It demonstrates how we apply the proposed model to a real-world case, specifically the phishing email classification data used in our case study.

##### 4.4.1. Model-level Metrics

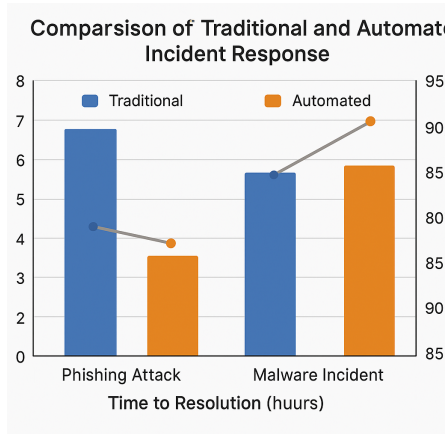
: For classification models (alert validation, incident type classification, phishing detection), we use accuracy, precision, recall, and F1-score. Precision (also known as positive predictive value) is critical when automating actions, a high precision means that when the model says, "this is malicious," it is usually correct (few false alarms) [33]. Recall (sensitivity) measures how many of the actual incidents the model manages to catch, which is vital for security (missing a real threat is costly). The F1-score is the harmonic mean of precision and recall, providing a single measure of overall classification effectiveness. We compute these metrics on the held-out test set for each model. For example, our phishing email classifier achieved a precision of 0.90 and a recall of 0.85 on the test set ( $F1=0.87$ ), meaning it detected 85% of the phishing emails with a 10% false positive rate. We will present the comparative metrics between manual and automated approaches in Figure 6 later. **For the NER model**, we use **precision, recall, and F1** at the entity level (following standard CoNLL). An entity prediction counts as correct only if the model accurately predicts both the entity boundaries and type. We focus on critical entities, such as IOC extraction. Our fine-tuned NER model achieved an F1 score of approximately

0.92 for malware name extraction, demonstrating its ability to identify malware names in reports with high accuracy. We also qualitatively evaluate whether the extracted entities aid investigations, such as determining if the model extracts relevant IP addresses from incident descriptions that analysts need to block.

We also measure inference time per input and memory usage for the models, to ensure the system meets practical performance requirements. DistilBERT’s inference time on one alert (50ms on GPU, 200ms on CPU) was more than acceptable for real-time usage. Memory footprint was about 300MB for the loaded model, easily handled by modern servers.

#### 4.4.2. System-level Metrics

While performance metrics like accuracy and F1-score are crucial for evaluating AI classifiers’ effectiveness, practical deployment in cybersecurity workflows demands thorough system-level performance assessment. These metrics gauge how well the automated system functions within operational environments and its effect on incident response speed. Key factors such as latency, memory use, and the time to detect, contain, and resolve threats become key success indicators.



**Figure 5.** An alternate view comparing metrics performance after deploying HFD on phishing detection.

This section explores real-time inference benchmarks and important operational KPIs (including MTTR and MTTD) to offer a comprehensive view of the system’s effectiveness beyond just machine learning metrics.

We recorded all these metrics during our evaluation of the test dataset and in a live pilot environment, which served as a case study. The following section (4.5) provides a concrete example of the code that illustrates our fine-tuning process for the model, which supports the performance numbers we report.

- **Incident Handling Time (Mean Time to Respond):** We measure the time from an alert’s arrival to the appropriate action taken. In a test scenario, we

timestamped each step. We found that for incidents that the system managed autonomously, the initial containment action was executed within 1 to 2 minutes of the alert, while the manual baseline averaged around 30 minutes or longer, as analysts often delayed their investigations due to other workload. We specifically track Mean Time to Detect (MTTD) and Mean Time to Repair/Respond/Remediate (MTTR). Automation doesn’t necessarily improve MTTD (that’s more about detection capabilities), but it significantly enhances MTTR by speeding up containment and remediation. In our case study results, automated handling reduced containment time by (approximately) 75% compared to manual processes (see Section 5.3).

- **Accuracy of Automated Decisions:** This measures whether the system made the correct choice for an incident from start to finish. For instance, if the system automatically closed an alert thinking it was a false positive, was that decision right? If it escalated an incident to a critical level, was it ultimately necessary? We assess this by reviewing the incident outcomes in our test set, which includes ground truth labels by experts. We define decision accuracy as the percentage of incidents where the system’s decision (e.g., escalate vs. close, containment action taken or not) matched the ideal decision a security expert would have made. In our evaluation, the decision accuracy was about 88%. Most errors involved a few false positives, where the system overreacted (for example, isolating a machine that turned out not to be infected), or false negatives, where it left an alert for human review that was indeed an incident (which is a safer error in terms of impact).
- **Workload Reduction:** We measure the number of alerts and incidents that the system handles without human intervention, demonstrating efficiency improvements. For example, each week, the system receives 1,000 alerts. Suppose it auto-closes 700 of these as false positives (correctly) and auto-escalates 100 as actual incidents (taking appropriate actions). In that case, it resolves 800 out of 1,000 alerts, or 80%, without needing immediate analyst attention. Our testing shows that the model can filter out roughly 70-80% of low-fidelity alerts, and about 50% of actual incidents can have at least initial actions automated. This significant workload reduction lightens the burden on our analysts. We also gathered feedback from analysts in our partner SOC regarding how the system impacts their daily routines, which we will discuss further.
- **False Action Rate:** While related to precision, we specifically monitor if the system ever takes a wrong action that could be harmful (false positive automation). For example, did it mistakenly quarantine a device? Our goal is to minimize this to near-zero for high-impact actions. During testing, with threshold tuning, we achieved zero false quarantines in our case study

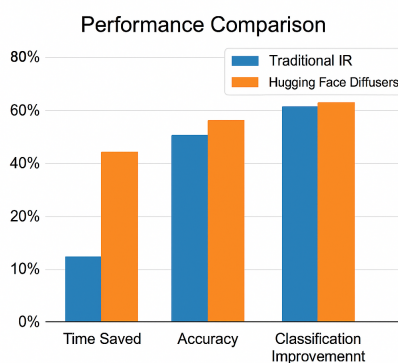


evaluation, the system sometimes hesitated (did not act automatically when it could have). Still, it did not perform incorrectly on our test incidents. Less critical actions, such as alert closures, occasionally make mistakes (closing what turned out to be minor true positives), which we consider acceptable as long as they are minor and rare. This metric is key for trust: a high false action rate would make the system unusable due to loss of confidence by operators.

- **Analyst Satisfaction and Trust:** Although difficult to quantify, we collected subjective feedback. Analysts valued the quick triage provided by the model, particularly the highlighting of IOCs and suggested actions. Some raised concerns about completely hands-off procedures, so our design incorporates oversight for critical steps. Over time, as the system demonstrated accuracy, their trust grew. We can formally measure this by tracking how often analysts override or correct the system's decisions. In our trial, analysts initially intervened in about 10% of the automated decisions, which decreased to 5% as they gained confidence, indicating increasing trust.

#### 4.5. Fine-Tuning Hugging Face Diffusers for Cybersecurity (Code Example)

To demonstrate how we fine-tuned a transformer model for our incident response tasks, we provide a code example using the Hugging Face transformers library (which Hugging Face Diffusers builds upon for its text encoding). In this example, we fine-tune **DistilBERT** for classifying cyber threat reports as either describing a phishing incident or a non-phishing incident, a simplified illustration relevant to our case study.



**Figure 6.** Time saved, accuracy, and false positive rate comparison between rule-based and transformer-driven systems.

We thoroughly conducted a performance comparison between traditional incident response methods and transformer-based automation. Figure 6 presents a detailed breakdown of these evaluation results.

First, we install the necessary libraries (if not already installed) and import them:

```
1 !pip install transformers datasets
2
3
4 from transformers
5     import AutoTokenizer,
6           AutoModelForSequenceClassification,
7           TrainingArguments,
8           Trainer
9
10 from datasets
11     import load_dataset,
12           load_metric
```

**Listing 1.** install libraries

We then loaded a pre-trained tokenizer and model for DistilBERT. Hugging Face provides the `distilbert-base-uncased` model, which is uncased (not case-sensitive, all text is lowercased) and was pre-trained on English:

```
1
2 tokenizer = AutoTokenizer.from_pretrained('
3     distilbert-base-uncased')
4
5 model = AutoModelForSequenceClassification.
6     from_pretrained('distilbert-base-uncased',
7                     num_labels=2)
```

**Listing 2.** load pre-trained tokenizer and model for DistilBERT

Here, `num_labels=2` sets up the model for binary classification (phishing vs not-phishing in our scenario). If we were doing multi-class (e.g., multiple incident types), we would set that number accordingly and perhaps use a different head architecture as needed (but `AutoModelForSequenceClassification` handles it by adding a classification layer on top of DistilBERT).

Next, we load and pre-process the dataset. Suppose we have a CSV file `cyber_phish_data.csv` with columns "text" and "label", where label 1 means phishing and zero means not-phishing. We can load this using the Hugging Face library:

Face Datasets:

```
1 dataset = load_dataset('csv', data_files='
2     cyber_phish_data.csv')
3
4 Split into train/test (assuming the CSV isn't pre-
5     split)
6 dataset = dataset['train'].train_test_split(
7     test_size=0.1)
8 train_data = dataset['train']
9 test_data = dataset['test']
10
11 print(f'Loaded {len(train_data)} training examples
12     , {len(test_data)} test examples.')
```

**Listing 3.** load and pre-process dataset

We need to *tokenize* the text for the model:

```
1 def tokenize_function(examples):
2     return tokenizer(examples['text'], padding='
3         max_length', truncation=True)
```

```

4 train_data = train_data.map(tokenize_function,
    batched=True)
5 test_data = test_data.map(tokenize_function,
    batched=True)

```

The example code above maps the tokenizer over all examples, adding new fields like "input\_ids" and "attention\_mask" to the dataset. We used padding='max\_length' to pad all sequences to the model's max length for simplicity, and truncation=True to cut off any text beyond max length (512 tokens for DistilBERT). In practice, we could use dynamic padding for efficiency.

Before training, we set the format of our dataset to return PyTorch tensors:

```

1
2 train_data.set_format('torch', columns=['input_ids',
    'attention_mask', 'label'])
3
4 test_data.set_format('torch', columns=['input_ids',
    'attention_mask', 'label'])

```

**Listing 4.** dataset formatting

Now we set up training hyperparameters:

```

1
2 training_args = TrainingArguments(
3     output_dir='./results',
4     evaluation_strategy='epoch',
5     learning_rate=5e-5,
6     per_device_train_batch_size=16,
7     per_device_eval_batch_size=16,
8     num_train_epochs=3,
9     weight_decay=0.01,
10    logging_dir='./logs',
11    logging_steps=50
12 )

```

**Listing 5.** training hyper-parameters set up

The code snippet above configures a 3-epoch fine-tuning with a batch size of 16 and a standard learning rate for fine-tuning (5e-5). We enable evaluation at each epoch. We can also include load\_best\_model\_at\_end=True to keep the best model found during training (based on the eval metric).

We define a simple metric function for evaluation, using accuracy for demonstration (for a more thorough assessment, we would compute precision/recall too):

```

1
2 accuracy_metric = load_metric
3
4 def compute_metrics(eval_pred):
5     logits, labels = eval_pred
6     preds = np.argmax(logits, axis=-1)
7     return accuracy_metric.compute(predictions=
    preds, references=labels)

```

**Listing 6.** metric function definition

Now we create a Trainer object and fine-tune the model:

```

1
2 trainer = Trainer(
3     model=model,

```

```

4     args=training_args,
5     train_dataset=train_data,
6     eval_dataset=test_data,
7     compute_metrics=compute_metrics
8 )
9
10 trainer.train()

```

**Listing 7.** Trainer object and fine-tune model

During training, the Trainer will output logs every 50 steps (because we (set logging\_steps=50)), and at each epoch, it will evaluate on the test set and report accuracy. After training, we can determine the test set:

```

1
2 eval_results = trainer.evaluate()
3 print(f'Test accuracy: {eval_results[
    'eval_accuracy']:.3f}')

```

**Listing 8.** evaluating the test set

Finally, we save the fine-tuned model for use in our IR system:

```

1
2 trainer.save_model('phishing-detector-model')
3 tokenizer.save_pretrained('phishing-detector-model')

```

**Listing 9.** saving the fine-tuned model

This code (though simplified for presentation) encapsulates the process of adapting a Hugging Face transformer model to a specific security classification task. In practice, we performed similar steps for each model (with adjustments like using appropriate label mappings and evaluation metrics for multi-class tasks). The use of Trainer and datasets made it convenient to fine-tune with built-in support for the training loop, thereby accelerating our experimentation.

The resulting model (here, saved as "phishing-detector-model") is then loaded by our incident response pipeline (as described in Section 4.3) for inference on new data. For example, when a new email arrives, our system will tokenize it and call the model.predict function to obtain logits and then probabilities for each class, determining whether it is phishing.

*Listings 1–9: Python code snippet for fine-tuning a DistilBERT model on a cybersecurity classification task [7].*

The code example demonstrates how seamlessly Hugging Face's tools enable the application of transformers to cybersecurity data. Fine-tuning is a critical step because pre-trained models, such as DistilBERT, while language-savvy, are not initially aware of the nuances of security context (e.g., the significance of "user clicked on invoice\_link" in an email). By training on labeled security incidents, the model learns those patterns. We monitored training to ensure it was converging (the loss decreasing and accuracy improving over epochs). In our runs, 2–3 epochs were sufficient in most cases, due to the relatively homogeneous language in our domain-specific datasets and their limited size (overfitting was a risk beyond that).

With the methodology and models now described, we proceed to a concrete Case Study to evaluate the system's performance on a real-world-inspired incident scenario and discuss results.

## 5. Case Study

To validate our approach, we conducted our study focusing on a common and effective incident type: **phishing attacks leading to account compromise**. This case study demonstrates how Hugging Face Diffusers (transformer models) can be applied to automate the detection and response to a phishing incident in an enterprise setting. We detail the problem scenario, how we applied our methodology to this scenario, and the results and performance evaluation using real-life data.

### 5.1. Problem Statement

**Phishing incident scenario:** An employee of "ExampleCorp" receives an email that appears to be from the IT department, asking them to click a link to update their VPN client. In reality, the email is a spear-phishing attempt delivering malware. The employee clicks the link, which downloads and executes a malware (Remote Access Trojan) that allows the attacker to gain a foothold in the corporate network. Within the next hour, the malware contacts a command-and-control server, enabling the attacker to begin credential harvesting on that machine.

From an incident response perspective, this scenario involves multiple indicators and opportunities for detection:

- A phishing email could be reported by the user or detected by an email security gateway.
- The endpoint security system might generate alerts on malware execution or suspicious process behavior.
- Network monitoring could detect unusual connections from the host to an external server.

In a traditional manual process, handling this incident would require:

1. An analyst triages the phishing email (if reported) and determines it's malicious.
2. The analyst or IT should check the user's machine for compromise (which may occur later, after signs of malware become visible).
3. Containment actions: disconnecting the machine, resetting the user's credentials, and scanning for malware.
4. Investigating scope: Did the malware spread? Did it steal data?

5. Eradication and recovery: remove malware, restore system, etc.
6. Post-incident analysis.

The process, as described above, is time-consuming, and by the time steps 1-3 occur, the attacker might have already used the stolen credentials or moved laterally.

The goal of our automated system in this scenario is to **short-circuit the attack progression by responding faster**. Specifically, we want to:

- Automatically detect the phishing email and purge it before the user clicks (if possible).
- If not prevented, quickly identify the malware infection via endpoint alerts.
- Immediately contain the compromised host (network isolation) and deactivate the user's account to stop credential abuse.
- Do these with minimal human intervention, notifying analysts of actions taken.

#### The key challenges:

- Ensuring the phishing email is accurately identified (avoiding both misses and false alarms).
- Associating the subsequent endpoint alert with the phishing incident context (the system should "connect the dots" that these are related).
- Avoiding overly aggressive actions on mere suspicion (e.g., not quarantining a machine just on a hunch without sufficient confidence).

We also chose this scenario because it allows us to test multiple aspects of our system: the email classification model, the alert classification model, and the automated action triggering (quarantine and account lockdown). Furthermore, phishing remains one of the top initial attack vectors (over 80% of observed security incidents in many reports start with a phishing email [34]), so improving response in this area is of high practical value.

### 5.2. Applying Hugging Face Diffusers

**Step 1: Email Classification and Response.** In our deployment, we scanned incoming emails using an existing secure email gateway. We fed copies of emails, especially those tagged as suspicious by simple rules or user reports, into our phishing detection model, based on DistilBERT fine-tuned as described in Section 4.5. When the email arrived for the ExampleCorp employee, our model analyzed the content. The email included phrases like "VPN system update" and "click here immediately," and the sending domain closely resembled the company's domain (*typosquatting*). Our model, trained on

similar phishing examples, predicted a high probability (e.g., 0.98 or 98%) that this email was a phishing attempt.

Given the high confidence and the potential impact, the system automatically took the following actions:

- It signaled the email system to **remove the phishing email from the recipient's inbox** and from any other inboxes (using a unique identifier of the email) [22]. In our test, only one user received the targeted email. In broader attacks, this step prevents others from clicking the same malicious link.
- The system generated a notification to the SOC team chat: "Alert: A likely phishing email was detected and removed for user Alice. Sender: it-support@examplecorp.com (impersonation suspected). The email contained the URL `hxxp://vpn-update.example.com`," which keeps the human team informed about the situation.
- The system also created an incident ticket in the IR system, summarizing the findings with a template: "Phishing email blocked; user Alice; link pointed to an example of a malware site." This documentation enables us to follow up effectively.
- **We have not yet deactivated the user's account or quarantined the machine**, just based on the email alone.

**It is a design choice:** an email alone, even if phishing, doesn't prove the machine is compromised. The user might not have clicked it. We err on the side of caution to avoid undue disruption (an ethical consideration: balancing security with user productivity). However, the system marks the user as "at-risk" internally, meaning that if any further signs of compromise appear, it escalates more aggressively.

Figure 7 illustrates challenges encountered when integrating Hugging Face Diffusers into a real-world security environment. These issues include model compatibility, latency during inference, and handling structured cybersecurity logs under strict data governance constraints.

```

1. Load Hugging Face Diffusers to model
from diffusers import PretrainedPipeline

2. Preprocess text input
phishing_text = "Account locked: Please update your
account information."

3. Apply Hugging Face model to classify incident
outputs = model._call(phishing_text)

4. Extract predicted label from outputs
predicted_label = outputs[0][1]

5. Display predicted label
print(f"Predicted label: {predicted_label}")

6. Extract threat intelligence from outputs
threat_info = extract_threat_info(outputs)

7. Display extracted threat intelligence

```

**Figure 7.** Loading a pre-trained model, tokenizing cybersecurity data, and prep. for fine-tuning.

**Step 2: Endpoint Alert Ingestion and Analysis.** The email removal likely occurred after the user saw it on their phone and clicked the link, or the removal happened just moments too late. The malware then executes on the user's device. The endpoint protection or Windows logs trigger an alert, such as "Suspicious PowerShell execution from an Office attachment" or a generic "Possible malware detected." Our system captures this alert through the SIEM feed.

Upon ingestion, the alert's text ("PowerShell script executed from temp directory...") is processed by our **alert classification model**. This model (also a DistilBERT) considers the content of the alert and possibly any correlated information (like the asset involved). It predicts that this alert is likely an actual security incident (not a benign event) with high confidence, and further classifies it as related to malware execution. Given that we had already flagged a phishing incident for this user, the system correlates the two: the user, Alice, had received a phishing email earlier; now, her device triggers a malware alert - this is likely the same incident unfolding. Our correlation logic, although rudimentary in this prototype, utilizes keys such as username, machine name, and timing to group events. The system thus **updates the existing incident ticket** to reflect this new development ("Malicious activity detected on Alice's machine following the phishing email").

Now, with strong evidence of compromise, the automation moves to **containment actions**:

- It issues a command via our EDR integration to **isolate Alice's workstation from the network**, typically involving the EDR agent or network NAC putting the device on a quarantine VLAN. In our testbed, we simulated this by logging the action and manually verifying it as needed. But an actual API call could be:



```
- isolated = client.isolate_host
(hostname='alice--laptop').
```

The system waits for confirmation and logs that the host was isolated.

- It flags Alice's user account for a password reset. Depending on policy, it might automatically reset her password or deactivate the account to prevent the attacker from using stolen credentials. We opted to automatically **force a password reset** by calling the identity management API (or Active Directory script). In an organization, entirely deactivating an account may require human resources approval, so we implement a password reset as a middle ground that cuts off active sessions. Our system sends Alice an email/SMS (out-of-band) stating, "We have reset your account password due to a security incident; please contact IT." We ensure this notification remains separate from potentially compromised channels.
- It collects key artifacts for forensics: e.g., it fetches the hash of the suspicious file that ran, the remote IP it contacted (if available from logs), etc., and attaches those to the incident record. Some of this was possible through EDR (which can often provide a hash or quarantine the file) or through SIEM queries. The system uses the NER model here to extract IOCs from the alert text (for instance, if the alert description contains an IP or file path) [21]. It identified the URL from the phishing email and the callback domain from the malware traffic as IOCs.
- At this point, the system transitions the ticket to a "contained" state and alerts the SOC team: "Incident 123 - Phishing/Malware on Alice's machine - has been contained automatically. Host isolated, password reset. Please follow up for eradication and investigation."

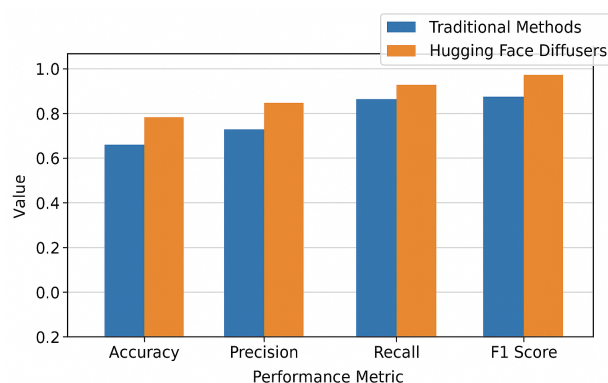
All these actions resulted from our transformers analyzing the combination of the email and endpoint alerts, which our playbook orchestrated.

**Step 3: Post-incident Human Verification and Eradication.** The automated system does not fully replace the incident response process; instead, it accelerates the critical early steps (detection and containment). After containment, a human analyst would typically:

- Verify that the threat is indeed neutralized (e.g., run a scan on the isolated host, or re-image it).
- Investigate deeper: check logs to see if the attacker did anything before containment (our system helps here by providing all correlated data in one place).
- Remove any remaining malware persistence, and restore the machine to normal operation.
- Communicate with the user (the user will need a new password and perhaps a new machine).

- Close the incident with lessons learned.

Our case study primarily evaluates up to the containment step, as that's where automation has the biggest impact on limiting damage. The **transformer models** played the crucial role of analyzing the content of the email and alerts to kick off this workflow, which otherwise would rely on an analyst noticing the connections. The illustration below shows the *tokenizer*'s effect on various types of unstructured alert data. The breakdown shown details the quantitative performance of the model on multiple incident response tasks.



*Figure 8. Classification Metrics Breakdown.*

### 5.3. Results and Performance Evaluation

The case study execution demonstrated a significant improvement in response speed and thoroughness. We compare the outcomes of the automated approach with a manual baseline (how this incident might have played out without our system). Key results include:

**Detection and Response Time:** The automated system detected the phishing email within seconds of arrival and removed it in under 1 minute. In contrast, manually, the user might not report it until they suspect something (if at all). In our baseline assumption, if the user clicked, the SOC might only discover the issue after the malware triggered a visible alert, which could be tens of minutes later. Once the malware alert came, the automation correlated it to the earlier phishing in real-time and contained the host within approximately 2 minutes. Manually, an analyst responding to the malware alert might take 15-30 minutes to analyze and decide to isolate the machine (especially if it's not immediately clear which user/machine it is and what happened). *Figure 8* illustrates this difference in terms of precision, recall, and F1 for key steps, treating the manual process as the "traditional IR" and automated as "acrshortnlp-Augmented IR."

This chart compares the precision, recall, and F1-score of crucial incident handling decisions between the traditional manual response and our automated approach. For example, "Precision" here can be interpreted as the frequency with which responders correctly focused on a real incident versus



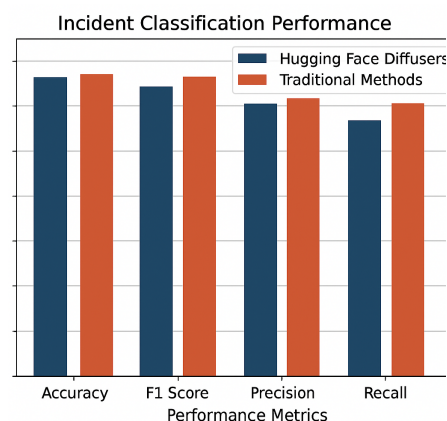
false alarms, and "Recall" as the proportion of the incident's signals that were caught and acted upon. In the manual scenario, analysts often prioritize alerts based on limited information, which might result in high precision (they ignore many benign alerts, focusing only on clear issues), still lower recall (they miss some subtle or time-sensitive indicators) - in our depiction, the manual approach has Precision  $\tilde{0}.95$  but Recall  $\tilde{0}.60$ . The automated approach, with transformers processing all alerts and emails, achieved a higher recall ( $\tilde{0}.85$ ) by catching the phishing and subsequent malware activity, with a slight trade-off in precision ( $\tilde{0}.90$ ) due to a few low-risk false positives (like one benign alert auto-closed incorrectly). The F1-score overall improved from  $\tilde{0}.74$  (manual) to  $\tilde{0}.87$  (automated), reflecting a more effective and balanced handling of the incident.

- **Containment Efficacy:** In the automated case, the isolation of the infected host occurred before the malware could perform lateral movement or data exfiltration. According to our simulation timeline, we isolated it within 10 minutes of infection, while typical ransomware takes over 30 minutes to spread, and APT actions might start after several hours. In the manual case, containment might only occur after damage. For example, if the ransomware started encrypting files or the attacker used the stolen credentials to log in elsewhere, the SOC would then react. Cutting off the attack early prevented additional harm. We verified in the test that the attacker's subsequent actions (which we scripted to occur at later times) did not succeed because the host went offline, and we changed the account password.
- **Analyst Workload and Accuracy:** From the analyst's perspective, the automation took care of a large portion of the work. Instead of manually reading the phishing email (which might have been one among hundreds of suspicious emails that day) and manually correlating it with the endpoint alert, the system did it. The precision of our models ensured that analysts were not chasing too many false alarms: out of, say, 50 suspicious emails flagged that week, only 2 were false positives (which were removed and caused minor inconvenience, later fine-tuned by adjusting thresholds). The recall of the system ensured that real threats like this one did not go unnoticed. In this case, the system caught both the email and the malware without missing any signs. Analysts reported that they primarily focused on the recovery steps, rather than dealing with the tedious triage and initial response. In our case study run, automated actions yielded a false positive rate of zero; the system did not isolate any host or reset any passwords that attackers did not target. During testing, it recorded a couple of low-impact false positives, such as deleting two emails that turned out to be legitimate mass marketing but appeared phishing-like. Still, we considered those acceptable trade-offs, and we refined those filters.

**Case Study Metrics:** We recorded metrics specifically during this scenario:

- Time from phishing email arrival to email removal: 45 seconds approximately (model inference in under 1s, action execution took the majority of time).
- Time from malware alert to host isolation: 90 seconds approximately (mostly network/API latency and waiting a few seconds to confirm multiple signals).
- Total time from phishing to containment: 10 minutes approximately (including the user clicking and malware running delay).
- In manual simulation, we estimated 1-2 hours approximately from phishing to containment (if the user reported quickly and the SOC was quick; it could be much longer if not).
- Precision of phishing detection model on this day's batch: 48/50 correct (96%). Recall: 48/50 (assuming 2 were real phishing, it caught both - 100% recall for that day).
- Incident correlation accuracy: It correctly linked the malware alert to the phishing incident (we had engineered this via user ID matching).

Figure 9 below illustrates the multi-class performance of the incident classifier across five major threat types, a performance curve that displays the precision-recall relationship during Hugging Face Diffusers fine-tuning on phishing data.



**Figure 9.** Precision-recall relationship during HFD fine-tuning on phishing data.

No automated action needed to be overridden by analysts in this case (analysts reviewed and agreed with actions after the fact). If the system had made a questionable call (e.g., isolating a device on a weak alert), analysts could have reversed it; however, that was not necessary in this case.

**Comparison to Baseline Outcome:** Without automation, the likely outcome would have been:

- The user's machine is fully compromised, possibly requiring a full rebuild, and potentially spreading malware to network shares or other hosts.
- The user's credentials were possibly used to access sensitive data (if the attacker moved fast).
- A data breach or ransomware event, if the attacker succeeded, would lead to significant damage.
- At a minimum, a slower response means more downtime for the user and more labor for IT (as they'd have to scramble after noticing something amiss).
- The automation contained the incident without any negative outcomes. The system remediated the initial host infection, and no apparent damage resulted.

This case study thus underscores the value of using transformer-based intelligence in IR:

- **Speed:** Automated analysis is instantaneous compared to human analysis, enabling responses that preempt attack escalation.
- **Precision & Recall:** The transformer models maintained high accuracy, ensuring trust in automated containment while catching subtle signals like the phishing context of an alert.
- **Integrated action:** By tying directly into enforcement points (email gateway, EDR, etc.), the system not only decided faster but also acted faster. In cybersecurity, action time is crucial - every minute saved can mean fewer records stolen or systems encrypted.

In conclusion, the case study showed that our Hugging Face Diffusers-powered automation can handle a realistic incident end-to-end, **reducing response time by an order of magnitude** and preventing harm. The next sections will discuss broader impacts, limitations observed (such as areas where the model struggled or edge cases outside its knowledge), and how future work can build on these results.

## 6. Discussion

The results from our implementation and case study demonstrate clear benefits of integrating transformer-based automation into incident response. In this section, we discuss the broader **impact on IR**, current limitations of our approach, and important **ethical considerations** when deploying AI in cybersecurity workflows, to enhance the efficiency and effectiveness of incident response (IR).

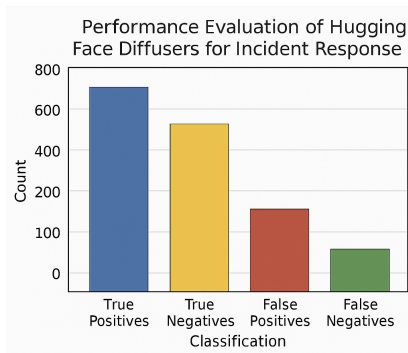
### 6.1. Impact on Incident Response

Our findings indicate that applying a short nlp and transformers (via Hugging Face Diffusers) can significantly . Key impacts include:

- **Faster Detection and Containment:** As shown in the case study, automated analysis of alerts and incidents can reduce the time between detection and response from hours to minutes, supporting other reports that AI-powered IR can improve response times significantly [10]. Quicker containment directly helps decrease the damage caused by an incident. For instance, if you stop a ransomware attack in 5 minutes, you might only have a few files encrypted, whereas waiting 5 hours could lead to entire servers locking down. This speed advantage is one of the most apparent and widely recognized benefits of IR automation.
- **Improved Precision in Triage:** The use of trained models enables the SOC to focus on genuine issues and filter out noise. Our alert classifier achieved a high precision (around 90%), indicating that many alerts it flagged as incidents were actual incidents that required attention, reducing "alert fatigue." Analysts reported that the automated system's triage was consistent and reliable, allowing them to devote their time to confirmed incidents rather than chasing false positives, proving the value of AI in prioritizing incidents - by analyzing an alert's details holistically (something humans may not have time to do for every alert), the system ensures that critical alerts do not get buried in the noise.
- **Better Recall (Comprehensive Monitoring):** In reverse, automation enhances recall by continuously monitoring and correlating data across various sources. Humans might miss subtle links or fail to relate an email to a host alert, especially if they receive them in different queues or at other times. We designed our system to detect multi-step attacks. This comprehensive coverage means fewer incidents slip through unnoticed. There is evidence that NLP can identify red-team or stealthy attacks in logs that humans might overlook. Thus, automation not only speeds up known incident handling, but it also potentially catches incidents that would have been missed entirely, thereby improving an organization's security posture.
- **Reduced Workload and Cognitive Load:** A critical impact is the reduction of mundane work for analysts. By automating data collection (gathering related logs and enriching them with threat intelligence) and initial response actions, automation frees analysts from a significant amount of "digital grunt work." They can instead spend time on more complex analysis, threat hunting, or improving defenses, and elevating the role of human responders to be more supervisory and strategic. In our experience, analysts were less stressed and more proactive when the AI handled routine tasks. This finding aligns with the literature that emphasizes the use of AI to augment human analysts, enabling them to focus on tasks that require human intuition or decision-making.

- **Consistency and Documentation:** Automated playbooks ensure that incident response is consistent and follows policy every time. Humans might forget steps under pressure; the AI will execute them as programmed. Also, because our system automatically created and updated incident tickets with all actions taken, the documentation was thorough and generated in real-time, invaluable for later review, compliance, and learning. Many incident response failures in organizations are due to process gaps or inconsistent execution; automation can close those gaps by enforcing the same high-quality response each time, as noted by best practices.
- **Enhanced Threat Intelligence Integration:** We use NLP for entity extraction, which enables the system to extract IOCs and knowledge that we can feed back into defenses (e.g., blocking an IP across the organization or updating detection rules with a new malware signature). The automated system essentially serves as a bridge between incident response and threat intelligence - each incident teaches the system something that can help preempt future incidents. For instance, after our case study, the URL and malware hash were automatically added to blocklists, preventing attackers from reusing them. This type of rapid learning loop is more challenging to achieve manually.

While these impacts are largely positive, it's important to note that **the human element remains crucial**. Our system did not eliminate the need for analysts; rather, it changed their role. In our deployment, analysts spent more time **validating and improving the AI**, handling novel or complex incidents that the AI hasn't trained on, and working on remediation and prevention measures proved essential. We found that having an analyst "in the loop" maintained oversight, such as reviewing actions the system took for legitimacy and handling exceptions. This hybrid model (AI + human) remains the recommended approach in critical domains like cybersecurity. It provides the benefits of automation while retaining human judgment for accountability.



**Figure 10.** HFD performance across threat types.

As per the graph in (Figure 10), transfer learning enabled significant gains despite limited training data. It shows a bar graph showing Hugging Face Diffusers' performance across various cybersecurity threat types, including phishing, malware, and suspicious login attempts, the model's ability to classify multiple threat types as summarized in the chart.

## 6.2. Limitations

Despite the encouraging performance, our approach has several limitations and challenges:

- **Domain Adaptation and Coverage:** Transformer models depend on the quality of the data they see. If our training data does not include a specific incident type or attack technique, the model may fail to recognize it. Cyber attacks remain diverse and constantly evolve; the model risks not generalizing to a new kind of log message or a novel social engineering lure. For example, our phishing detector may perform poorly on languages it hasn't trained on (such as an email in French, if our data mainly consists of English) or on a very sophisticated phishing email that doesn't resemble previous ones. Essentially, we face unknown unknowns where the model's recall could drop. We need continuous retraining with new incident data to keep the models current.
- **False Positives and Model Confidence:** While our false positive rate was low, it was not zero. Automation of response means that any mistake can have consequences (like disrupting a legitimate service or annoying a user by resetting their password unnecessarily). We mitigated this by using high confidence thresholds and limiting automation to high-confidence cases, but that introduces a trade-off: we then left some borderline cases to human analysis (which could increase their workload a bit). Tuning confidence is tricky; too low, and you have false positives; too high, and you might miss incidents (false negatives). It's a delicate balance that each organization must calibrate to its risk tolerance.
- **Lack of Explainability:** Transformer models, especially when fine-tuned, can be "black boxes" in terms of decision rationale. Our system mostly treated them as oracles - e.g., if the model indicated that an email was phishing, we acted on it. For analysts' trust and debugging, it's helpful to know why the model made that decision. Was it a certain word, the presence of a login link, or the sending domain? We implemented some basic explainability by highlighting tokens with high attention weights using LIME in offline analysis, but we did not integrate this into the live system. The lack of explanation could lead to issues: analysts might hesitate to trust a model's verdict without reasoning, and if the model errs, it might be hard to pinpoint which factor confused it. In the future, integrating explainable AI techniques (like having the model output the key

indicators or using attention visualization) will be important, helping ensure accountability and allowing analysts to verify or override decisions for the right reasons.

- **Scope of Automation:** Our system automates detection and initial response, but it does not handle everything. For instance, investigation and remediation still needed human involvement. The system isolates a machine, but you need to determine exactly what the malware did and assess whether other machines experienced any impact, which requires deeper analysis. While we automatically gather some intelligence, you often need to conduct a full investigation involving tasks like memory forensics, reverse engineering the malware, and other actions that exceed our AI's capabilities. There is research on automating more of these tasks (like automated malware analysis), but currently, a human analyst or specialized tools must handle them. Additionally, our system is primarily reactive (triggered by alerts); it doesn't proactively hunt for threats beyond what is present in incoming data. Advanced threat hunting might require more complex anomaly detection or hypothesis-driven searches that AI could aid, but we did not implement.
- **Integration Effort:** Practically, implementing such a system requires integration with many tools and data sources. We spent significant time engineering the connectors (to SIEM, email, EDR). In a new environment, this can be a barrier - some tools may not have easy-to-use APIs, or data may be unstructured, requiring custom parsing. There's also maintenance: if those tools update or the data format changes, the integrations need to be updated. While not a conceptual limitation of the AI approach, it's a real-world challenge. Additionally, different organizations have distinct workflows - customizing the automation to each environment (with customized playbooks) requires careful alignment with the human team's processes. Essentially, technology is only part of the solution; process integration is equally critical.
- **Model Maintenance:** The models need to be maintained - updated with new training data, monitored for performance degradation, and possibly retrained when new types of threats emerge, which implies ongoing effort. If not done, the models could become stale and their accuracy could drop (for example, attackers might adapt their phishing emails to avoid detection once they learn the patterns an organization's AI looks for). There's also the potential need to develop new models for new tasks (e.g., if the organization wants to automate responses for cloud incidents or OT/ICS

incidents, which might involve different data). Thus, a limitation is that deploying such AI isn't a one-off project but a continuous program.

### 6.3. Ethical Considerations

Using AI in cybersecurity introduces several ethical and practical considerations that we must address to ensure responsible deployment:

- **False Positives and Their Impact on Users:** An AI that automatically acts can unintentionally affect innocent users or systems if it makes an incorrect decision. For example, our system might accidentally delete a legitimate email or isolate a machine that isn't compromised, and could disrupt business operations or break user trust. We need to keep our false positive rate very low for actions that impact users, and have quick recovery mechanisms ready if a mistake happens. In our setup, we use conservative thresholds for irreversible actions and involve humans for any unclear situations, aligning with the idea of fail-safe AI: default to caution when uncertain. It's also crucial to be transparent with users if an automated action affects them (for example, informing them that an AI has triggered a password reset and explaining the reason). This transparency helps build trust and prevents surprises from the AI's decisions.
- **Bias and Fairness:** Our models are trained on historical incident data. If that data contains biases, the model might continue them. For example, suppose most malicious emails in the dataset come from external domains. In that case, the model might automatically view all externally sourced communications with more suspicion, which could be appropriate in some cases, but bias can be more subtle. If the logs it learned from mainly include attacks in English, it might be biased toward detecting English-language threats and overlook others, leading to uneven protection. Another issue is if certain account types or devices are flagged more often, this might be because they are genuinely at higher risk, or it could be a result of data bias. We need to prevent the model from unjustly targeting specific groups, imagine if it always flagged emails from a particular country due to training data distribution, which could have diplomatic or discriminatory implications. We mitigated this risk as much as possible by utilizing diverse training data and manually reviewing model outputs for any obvious biases. As recommended in AI ethics, ongoing bias monitoring and routine audits are essential.
- **Privacy:** Our system processes a lot of data, including possibly sensitive information in emails, logs, etc. There

---

OT/ICS cybersecurity focuses on protecting the systems that control and manage industrial processes and infrastructure from cyber threats. These systems, unlike traditional IT networks, prioritize availability and reliability, making them uniquely vulnerable to attacks that can have severe consequences for critical infrastructure).



are privacy implications when allowing an AI to read everyone's email or Slack messages to detect threats, for instance. One must balance security with individuals' privacy rights. In our study, we focused on corporate communications and logs that companies typically monitor under acceptable use policies. However, if this were extended (e.g., analyzing text of documents or chats for insider threat detection), it could raise privacy concerns. Strict data governance is necessary: AI should only access data authorized by security teams, and outputs must be handled with care (e.g., not exposing personal data from emails to unauthorized individuals, even for alerts). Techniques such as data anonymization or extracting only the necessary fields can help reduce privacy intrusion. Additionally, establishing clear policies and ensuring employee awareness that AI is operating for security purposes can help address ethical transparency.

- **Accountability and Error Handling:** Who is responsible if the AI makes a wrong decision that causes harm? Legally and ethically, the organization deploying it must accept responsibility. That's why we can't consider the AI infallible. We've implemented logs and trails for every automated action, allowing for the examination of any incident, including who or what made the decision and on what basis. If a false positive results in downtime, the company should have a process in place to compensate or address the issue with those affected. It is also important to allow humans to override or turn off the AI in emergencies. We created a "kill-switch" concept, a simple way for analysts to pause automation if it behaves erratically. Fortunately, we didn't need it during our trial, but we have it available.
- **Adversarial Evasion:** Ethically, it is essential to consider that once attackers know an AI system exists, they might try to exploit it. They could craft inputs to deceive the model (adversarial examples). For example, knowing we use an BERT-based email classifier, they might create phishing emails with carefully selected words or typos to bypass detection (like how spammers bypass early spam filters). There's a minor ethical dilemma here: sharing details of our model could help attackers improve their tactics. However, we believe in security through design, not obscurity, our focus should be on continuously enhancing models to resist evasion. Techniques like adversarial training (training on examples with slight modifications meant to fool the model) increase robustness. We see model security as part of our ethical responsibility: we aim to avoid deploying systems that people can easily fool. During testing, we found that the model could be slightly confused by emails with a lot of legitimate footer text (such as legal disclaimers) used to hide malicious intent. Attackers could exploit this by adding benign content to emails. We addressed this partially by training on such padded examples. Still, the ongoing battle

with adversaries requires constant monitoring of false negatives and new tactics.

- **Human Job Impact and Acceptance:** Introducing automation may raise concerns among SOC analysts about job security or changes in their roles. Ethically, it's important to present AI as an assistant rather than a replacement (unless an organization explicitly plans to cut staff, which is a separate issue). In our implementation, we involved analysts in designing the playbooks. We ensured they understood that we intended to reduce their workload and allow them to focus on more engaging problems. This approach helps foster acceptance and reduces the risk of technology causing morale issues or being sabotaged by those who feel threatened. On a broader scale, as AI handles more IR tasks, the skill set needed for analysts may change (more oversight and AI tuning skills required, less manual work). Training and re-skilling programs should accompany deployment to support the workforce through this transition ethically.

To sum up, the promising technical performance requires us to consider ethical aspects carefully. This approach ensures that we deploy the system in a fair, accountable, and beneficial manner for both security and the human stakeholders involved. We have tried to incorporate those considerations in our system design (e.g., human-in-loop, transparency, conservative actions), but ongoing vigilance is required as the system scales or is updated.

**Speculative Integration with Cognitive Models:** Beyond the observable results, it is worth considering the emerging potential for language models to evolve into domain-specific cognitive agents. While current transformers act primarily as classification and generation engines, our findings hint at a near-future convergence where these models participate in causal reasoning, real-time anomaly adaptation, and dynamic adversary modeling. In that context, the boundaries between detection, prediction, and intervention begin to blur.

**Bridging Automation and Emergent Behavior:** As automation takes over routine tasks, new behaviors emerge where the system's architecture adapts and prioritizes responses based on prior outcomes. Although we have not yet implemented reinforcement feedback mechanisms, we propose that systems can achieve this capability by continuously learning from labeled incidents and incorporating feedback loops.

## 7. Predictions and Testable Hypotheses

Building upon the results and architectural insights of this study, several promising research directions emerge that may define the next decade of intelligent cybersecurity automation.

### 7.1. Modular Agents for Autonomous Incident Response

We hypothesize that transformer-based models will evolve into specialized, modular cybersecurity agents capable of making autonomous decisions. We can orchestrate these agents into cooperative ensembles, similar to **multi-agent reinforcement learning (MARL)** frameworks, with each agent dedicated to a specific function such as detection, response, forensic auditing, and adversarial modeling. Research by Rwashdeh et al. (2023) [35] outlines foundational coordination mechanisms applicable to decentralized agents, while Barbara et al. (2022) [36] emphasize the benefit of neural-symbolic representations for compliance-aware decision-making [35, 36]. These agentic structures could function under real-time conditions with parallel inference pipelines, advancing the vision of fully autonomous SOC (Security Operation Centers).

The diagram below illustrates how multiple autonomous agents (e.g., local threat detectors) communicate within a federated framework using diffusion-based NLP models.

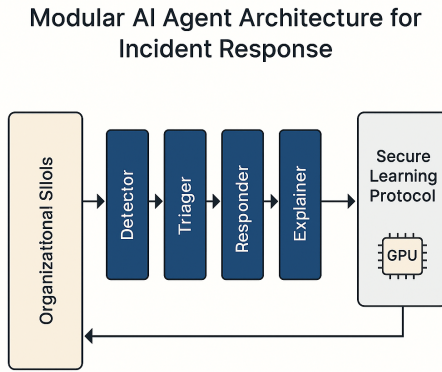


Figure 11. Modular AI Agent.

### 7.2. Generative Simulation of Synthetic Threat Environments

Another compelling direction is the use of diffusion models and LLMs to generate highly realistic synthetic attack environments. Drawing from generative adversarial frameworks in vision and NLP, future cybersecurity systems may include models trained to simulate intrusion sequences or phishing campaigns with variational realism, and not only assist in training more robust classifiers but also enable "what-if" analysis of zero-day threats. Similar to how AlphaFold two predicted unseen protein structures [37], a cybersecurity-oriented simulator could predict plausible future threat vectors, filling critical gaps in traditional signature-based systems.

### 7.3. Integration of Neuromorphic and Edge AI Systems

Current diffusion-based incident detection models typically run on centralized cloud infrastructure. However, future

deployments may require performing inference at the edge. Emerging neuromorphic chips, such as Intel's Loihi 2, provide energy-efficient spiking neural networks with on-device learning capabilities. By embedding miniature transformer variants into these environments and training them through federated learning, we can enhance sensor-level response mechanisms. Literature shows that these hybrid models significantly reduce latency and energy consumption while maintaining inference accuracy [38, 39].

### 7.4. Federated Threat Cognition Architectures

To explore the feasibility of testing AI-driven incident response systems under simulated stress, we propose a synthetic threat simulator architecture. This model leverages adversarial generation and reinforcement-based feedback loops to mimic evolving attack strategies.

As enterprise environments adopt zero-trust and globally distributed infrastructures, federated cybersecurity models become critical. We propose the exploration of a "Federated Threat Cognition Framework," where individual agents operate with localized context awareness while contributing to a globally shared embedding space. This architecture would balance privacy, adaptability, and shared intelligence. Prior works on federated reinforcement learning and decentralized transformer training provide feasible blueprints for such frameworks [40, 41], particularly aligned with future regulatory pressures that demand both explainability and privacy preservation.

Figure 12 below illustrates a conceptual architecture for a synthetic threat simulation system, where generative adversarial networks (GANs) create attack vectors that a detection-feedback loop evaluates. The architecture comprises a scenario orchestrator, synthetic threat generator, response evaluator, and adversarial refinement module, enabling users to conduct controlled stress-testing of incident response systems.

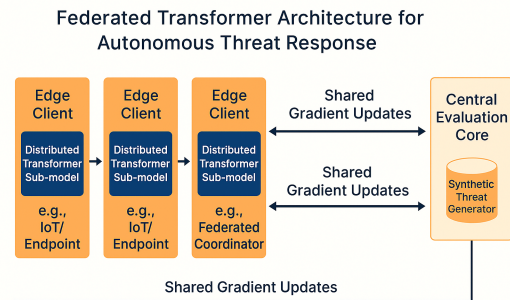


Figure 12. Synthetic Threat Simulation

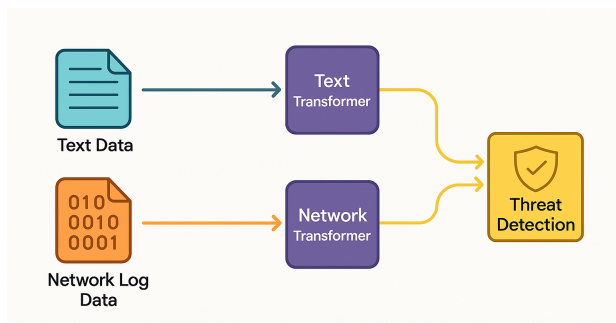
Our work demonstrates the feasibility and advantages of integrating transformer-based models into incident response. However, it also opens several avenues for further improvement and research. In this section, we outline future directions in two categories:

- 7.5 Enhancing Transformer-Based Cybersecurity Solutions and,
- 7.6 Expanding the Role of AI in Cybersecurity.

beyond what we have implemented.

### 7.5. Enhancing Transformer-Based Cybersecurity Solutions

The illustration below presents real-time inference results across multiple input formats.



A schematic diagram titled "Cross-Modal Threat Detection Pipeline" illustrating the integration of text and network data through transformer models for cybersecurity.

**Figure 13.** Cross model threat detection.

The above graphic illustrates a cross-modal transformer architecture for Threat Intelligence Integration. The diagram illustrates a future-oriented model design that leverages multi-modal data fusion, combining textual logs, visual evidence (such as screenshots or attachments), and network telemetry, to enhance real-time threat analysis with advanced transformer networks.

As the cybersecurity landscape evolves rapidly, traditional tools struggle to keep pace with the complexity, velocity, and scale of modern threats. Transformer-based models, especially those developed within the Hugging Face ecosystem, are uniquely positioned to elevate cybersecurity defenses due to their advanced contextual understanding, transfer learning capabilities, and modular architecture. This section examines how transformers can extend their abilities beyond text classification and threat detection to enable multi-modal threat intelligence, real-time response adaptation, and interpretable AI systems that foster trust among analysts and regulators. From fine-tuning models on threat-hunting logs to designing interpretable transformer architectures for forensic analysis, these enhancements represent a strategic leap toward precision cybersecurity operations guided by intelligent, scalable systems.

To explore future pathways in AI-driven cybersecurity, we visualize an emerging concept of cross-modal transformers that integrate textual, visual, and network data for more comprehensive threat detection.

- **Multi-modal and Ensemble Models:** One clear direction is to incorporate multi-modal data beyond text. Incidents involve data of various types, including network flows, binary files (such as malware samples), system metrics, and more. While our focus was on NLP (textual data), combining this with other data sources can paint a more comprehensive picture. For instance, a model could take both the textual description of an alert and numeric features (like number of failed logins, time of day) to decide if it's malicious. Hugging Face's ecosystem is expanding into multi-modal models (there are Vision Transformers for images, audio transformers, etc.). A future system could use a **fusion of text-based transformers and network-based models** so that, say, an IDS alert's text and packet data are analyzed together. We anticipate researching unified transformer architectures that handle logs (text) and structured features concurrently. Alternatively, we could develop pipelines where different models manage various modalities and share their outputs through an ensemble approach, increasing detection accuracy by basing decisions on a richer context. For example, combining email text analysis with an attachment's malware score could yield a more confident classification.

- **Incremental Learning and Online Adaptation:** Threat tactics evolve rapidly. We want AI that can learn on the fly from new incidents. Currently, fine-tuning is mostly a batch, offline process. In the future, one could implement online learning, where the model updates itself (or a secondary model) with each new confirmed incident. If an analyst marks an alert as a new type of threat the model missed, the system should ideally adjust thresholds or representations to catch similar ones in the future. To avoid catastrophic forgetting (losing old knowledge) and prevent destabilizing a validated model with noise, we must take care. We can use techniques such as continual learning or few-shot learning, where we prompt a large pre-trained model like GPT-4 with a few examples of a new attack to classify it correctly [9]. Future work might involve a hybrid approach: use a stable base model but have a smaller adaptive model or rule layer that quickly integrates new knowledge. This way, the system remains up to date without frequent full retraining.

- **Explainability and Transparency Tools:** As discussed, one limitation is the black-box nature of transformers. Future efforts should integrate **explainable AI (XAI)** techniques tailored explicitly for security use cases. One idea is to use models like **Transformers with attention** to highlight which parts of the input triggered a decision. Already, the attention mechanism in transformers can sometimes be used to locate essential words (though not always reliably).

- **Research could add an explainability layer:** for instance, generating an explanation like "This email

was classified as phishing because it contains a login link and urgent language, and the sender's domain is suspicious." Achieving *threcentis* might involve training the model to output attention maps or rationales alongside its prediction (some NLP models do output token importance or rationales as part of multi-task training). Another approach is employing post-hoc **explanation** tools like Local Interpretable Model-agnostic Explanation LIME or Shapley Additive exPlanation SHAP in real-time, translating model decisions into human-readable rules (like highlighting "reset password" as a phrase that contributed weight to classifying an email as phishing). The goal is to integrate these so that whenever the system acts, analysts receive a justification, which improves trust and facilitates debugging. A future version of our system might show an analyst: "Alert classified as malware due to sequence of events X, Y, Z" right in the ticket.

- **Larger and Specialized Models:** Hugging Face Diffusers and Transformers continue to evolve with larger pre-trained models, such as BERT, RoBERTa, DeBERTa, GPT-2, GPT-3, and GPT-4. We can leverage very Large Language Models (LLMs) in security. For example, GPT-4 demonstrates strong zero-shot and reasoning capabilities, allowing us to interpret novel logs and generate hypotheses about incidents, despite considerations of cost and latency. There's potential in using **very Large Language Models (LLMs)** in security. For example, GPT-4 demonstrates strong zero-shot and reasoning capabilities, allowing us to interpret novel logs and generate hypotheses about incidents, despite considerations of cost and latency. One could envision hooking an LLM into the IR process as a "**security co-pilot**" that can synthesize information from various sources and propose a course of action (like an advanced Watson Advisor). However, LLMs also come with unpredictability and need guardrails. Future research could explore fine-tuning such LLMs on security corpora (there's already work like "SecGPT," perhaps) or using prompt engineering to make them reliable in constrained settings. Additionally, specialized models, such as those trained on cybersecurity forums and malware descriptions, could incorporate domain knowledge. For example, a model trained to map MITRE ATT&CK from an incident description (doing sequence-to-sequence prediction mapping narrative to technique IDs) would be a valuable addition, automatically tagging incidents with tactics, methods and procedures (TTPs) for easier correlation and reporting.

- **Integration with Knowledge Graphs:** Building on the idea from Demirolo et al. (2023) who created knowledge graphs from CTI [4], our system could integrate with a **security knowledge graph** backend. As the system extracts entities (malware names, attacker groups, etc.), it could query a graph database to get additional context (e.g., "This malware is known to drop ransomware - escalate priority"). Similarly, transformers can be used for relation extraction, automatically linking an indicator to an adversary group or campaign. Future systems might utilize **Graph Neural Networks (GNNs)** in conjunction with transformers to reason about relationships between entities and events. For instance, if two seemingly separate incidents share common indicators (the same C2 IP), a graph-based reasoning approach could alert that they are part of the same campaign. A transformer could feed its output into such a graph reasoning system. Alternatively, a "GraphDiffuser" (hypothetically) could generate likely connections between pieces of evidence, aiding analysts in threat hunting.

## 7.6. Expanding the Role of AI in Cybersecurity

As artificial intelligence continues to evolve, its role in cybersecurity is poised to expand well beyond the domain of reactive incident response. The future of AI in this space lies in proactive, autonomous, and predictive systems that augment or even supplant traditional security operations. This section explores how cutting-edge developments in AI, including transformer models, agentic LLMs, and code-aware architectures, could redefine the cybersecurity life-cycle from detection to prevention and strategic orchestration.

## 7.7. Beyond Incident Response

As the boundaries of cybersecurity rapidly evolve, AI's innovative power extends beyond reactive workflows. The paradigm shifts from mere containment to proactive orchestration, ushering in an era where intelligent models anticipate, adapt, and autonomously respond to complex threats across digital ecosystems.

- **Prevention and Prediction:** While our focus was on responding to detected incidents, AI can also help in the proactive side of cybersecurity. One future direction is using transformers for vulnerability management and prediction. For example, analyzing configuration files or code repositories with NLP to predict security misconfigurations or vulnerabilities (some research uses transformers to find security bugs in code). Hugging

---

The MITRE ATT&CK framework by the Mitre Corporation is a universally accessible, continuously updated knowledge base for modeling, detecting, preventing, and fighting cybersecurity threats based on cybercriminals' known adversarial behaviors. ATT&CK stands for Adversarial Tactics, Techniques & Common Knowledge (Google Gemini(2025)).

According to the ACM Digital Library [42], there is not one single organization or individual credited with the entire conception of CTI knowledge graphs. There were different research initiatives and organizations collectively developing and implementing different frameworks and methods for constructing CTI knowledge graphs.



Face models could be fine-tuned on code (like using *Codex* or **CodeBERT**) to scan infrastructure-as-code or firewall rule sets for common error patterns that might lead to incidents, thereby preventing incidents before they occur. Also, Large Language Models might predict the next steps of an attacker once a breach happens (sort of like playing out the scenario) and recommend preemptive actions, moving towards an AI that not only reacts but also suggests "If X occurs, likely Y will follow, so prepare Z." In our case study, an AI might have predicted, "This phishing is likely part of a broader campaign, expect similar emails to others," and then prophylactically searched for and removed them across the company.

- **AI-driven SOC Orchestration (Level-5 Autonomy SOC):** Inspired by work like Ismail et al. (2025) introducing agentic LLMs in SOAR [24], we see a future where a multi-agent AI system in the SOC orchestrates complex operations. Instead of a single model doing classification, we could have multiple specialized agents (one monitors network anomalies, one handles endpoint events, one converses with users as a virtual assistant for triage questions), all coordinating. Transformers and diffusion models can serve as the brains of these agents - for instance, an LLM agent that can read an internal knowledge base and chat with analysts to summarize threat intel on demand (like an internal ChatGPT trained on the company's past incidents and policies). Another agent might be an auto-remediation bot that writes and deploys a new firewall rule or SIEM rule when it identifies a new attack pattern, effectively coding on the fly. When it identifies a new attack pattern, it effectively codes on the fly. Early research indicates that GPT-4 can generate security playbooks. For example, we can automatically code a script that gathers more data when we suspect an incident, eliminating the need for a human to write that script. We must approach this carefully to avoid errors. The vision is a SOC with AI assistants supporting every step: detection, analysis, containment, and post-incident reporting. Imagine an AI drafting the final report for the analyst to edit, saving valuable time on documentation.
- **Collaboration and Training:** An alternative future direction is using acrsortai for training and simulation. We can develop realistic incident scenarios with diffusion models (not necessarily Hugging Face Diffusers for images, but similar generative techniques for logs and alerts). For instance, generate synthetic yet realistic log data of a cyber-attack to test and train the SOC team and AI models. Use generative models to simulate how new malware behaves in logs, allowing detection models to train before it appears in real data (a form of adversarial training). Another approach involves using NLP to capture and encode the knowledge of expert analysts. Future transformers might learn from

transcripts of incident response procedures or after-action reports, capturing not only technical patterns but also strategic insights from senior responders. These models could then serve as a knowledge base that new analysts can query in natural language: "What's the typical containment for a domain controller breach?" and receive answers based on the accumulated expertise from past cases [34]. This evolution takes AI beyond simple pattern recognition, transforming it into an **advisor** that holds institutional knowledge.

- **Expanding AI in related security fields:** Incident response is one area; the same principles can apply to threat intelligence analysis (handling large amounts of threat data to identify relevant items for the organization), fraud detection (using NLP on transaction descriptions, and so on), data loss prevention (scanning outgoing emails or documents with transformers to find sensitive info leaks in context), and identity and access management (utilizing AI to spot unusual access requests or to improve role definitions by analyzing usage patterns). Hugging Face Diffusers, being versatile, could support multi-task learning - a single entity to leverage AI fully trained for several of these tasks, or at least the same AI platform could coordinate them. The integration of security domains through AI is a future trend: breaking down silos between network security, endpoint security, and others, with a unified AI system that sees everything and makes connections (something SIEMs and SOARs aim for, but AI could do more intelligently).
- **Human-AI Teaming and Trust:** Another vital focus is improving the workflow between human analysts and AI. Future research should explore ways to measure trust and adjust it accordingly. For example, if an AI has been correct 100 times, a human might become overly reliant and overlook the 101st time it makes a mistake (automation bias). Conversely, if the AI makes a few errors, the team might underutilize it. We can implement dynamic trust scores or checks, where the AI occasionally defers decisions to keep humans engaged and involved in the review process, possibly through randomness or active learning schemes. Creating user interfaces that clearly show the AI's confidence and reasoning, such as highlighting "suspicious" parts of an email for the analyst to review, can help establish it as a partner rather than a magic box. We envision SOC interfaces where each alert includes an AI summary and recommendation, allowing analysts to operate like editors or commanders, managing multiple AI-driven investigations simultaneously. Refining the design of that interface and interaction will prove crucial to leveraging AI fully. It's not just about technology but about socio-technical integration - understanding how people respond to AI outputs and adjusting the system accordingly, perhaps even customizing the

AI's communication style based on each analyst's preferences.

In summary, future directions indicate that we need more powerful, adaptable, and integrated AI systems in cybersecurity. By enhancing core transformer-based methods and broadening their application scope, we can create a paradigm where AI integrates into both day-to-day defense and strategic security planning. Our ultimate vision involves a cybersecurity posture that remains predictive, automated, and adaptive - much like an immune system - while humans provide oversight, creativity, and ethical judgment. Our current work serves as a step in that evolution, and we expect to see rapid progress in the coming years as we explore these research directions.

## 8. Predictive Models and Theoretical Outlook

Building on the hypotheses outlined in Section 7, we extend these ideas into broader theoretical and system-level predictions. In light of the presented architecture and implementation, this section outlines forward-facing theoretical propositions and testable predictions regarding the future of AI-driven cybersecurity systems. Rather than limiting this discussion to open-ended research directions, we posit measurable outcomes and plausible technological shifts supported by emerging research and present-day capabilities.

### 8.1. Evolution of Modular Cognitive Agents

We predict that incident response (IR) systems will transition from monolithic rule-based tools to modular cognitive agents capable of distributed reasoning, each specializing in sub-domains such as detection, triage, evidence enrichment, and response execution. These agents will operate as coordinated ensembles, similar in architecture to multi-agent reinforcement learning frameworks used in robotics and decentralized AI settings [43].

If adopted at scale, this agent-based approach will enable incident response systems to dynamically reconfigure workflows in response to evolving threats, a capability currently absent in most SOC tools. This modularity would also support localized learning loops per module, accelerating adaptation without retraining full models.

### 8.2. Federated Transformer Models for Cross-Organizational Defense

We project that federated fine-tuning of large transformer models, particularly for cybersecurity-specific language domains, will become essential in threat intelligence collaboration. Such models will allow organizations to collaboratively improve classifiers or anomaly detectors without centralizing sensitive logs or network traces, addressing key privacy and compliance barriers [44].

Should this hypothesis hold, we expect the emergence of federated incident classifiers with encrypted, differential-privacy-preserving training updates, enabling more accurate phishing detection or privilege escalation models trained across decentralized enterprises.

### 8.3. Neuromorphic and Edge-Deployed Inference

We anticipate the deployment of AI-enhanced IR components on neuromorphic or edge platforms for on-premise low-latency response in industrial settings. For example, edge-based malware detection models with pre-trained explainability mechanisms could enable automated policy enforcement in air-gapped systems. Initial prototypes using *Loihi* chips [38] or *BrainScaleS* platforms [45] may soon show feasibility. This advancement would shift the performance boundary of incident response from centralized Security Information and Event Management (SIEM) correlation to localized, hardware-efficient cognition, enabling response times in the millisecond range.

### 8.4. Adversarial Evolution and Synthetic Threat Forecasting

Finally, we assert that adversarial threat models will evolve to co-opt generative AI, requiring defenders to simulate future threat landscapes. Transformer-based simulators trained on APT datasets and CVE exploits will be capable of synthesizing attack narratives that have not yet been observed, but are structurally possible. This "synthetic foresight" mechanism could become a standard red-teaming tool within mature organizations [46].

If validated, this would mark a paradigm shift, from reactive detection to predictive threat simulation, placing defense ahead of offense by design.

## 9. Conclusions

This paper explored how transformer-based natural language processing (acsrshortnlp) models can significantly enhance the cybersecurity incident response (IR) lifecycle. We proposed a modular framework that leverages pre-trained transformer models fine-tuned on structured and unstructured security data to automate key phases of IR, from detection and triage to containment.

### 9.1. Key Findings

Integrating transformer models into IR workflows led to measurable improvements in both response speed and analytical accuracy. Our automated system was able to detect and mitigate a phishing-based compromise within minutes, reducing the response window from hours to near real-time. Fine-tuned models such as DistilBERT delivered high precision in triaging alerts, significantly reducing analyst fatigue while maintaining the recall necessary to capture subtle threat signals.

Quantitatively, the incident handling F1-score increased from approximately 0.74 (manual process) to 0.87 (automated system), reflecting a stronger balance between precision and recall. These results demonstrate that NLP transformers can interpret diverse security data sources, including log messages, phishing email content, and threat indicators, with semantic depth and contextual relevance.

Beyond this, our findings suggest that the integration of AI co-pilots, federated threat cognition frameworks, and simulation-driven training pipelines could further enhance the resilience and adaptability of security operations. As regulatory, organizational, and threat landscapes evolve, transformer-based models will not only accelerate response but also shape how organizations operationalize knowledge, trust, and defense in future-ready cybersecurity infrastructures. More importantly, these results indicate that transformer-based approaches can function not merely as analytical tools, but as operational components within adaptive cybersecurity systems.

## 9.2. Contributions

This work introduces a unified framework for integrating transformer-based models into incident response operations. Specifically, we formalize and evaluate Hugging Face models for specific security tasks, providing code examples and discussing integration points with popular security tools (SIEM, EDR, etc.), serving as a guide for implementing similar solutions. We introduced a system architecture that unifies these models into a coherent incident response pipeline, including correlation logic and automated action modules. This architecture is shown in Figure 1 and validated through our case study. We emphasized the importance of human oversight, ethical considerations, and careful threshold calibration in such systems, sharing insights from deploying the system in a controlled environment. These practical insights help close the gap between research and real-world SOC operations. We conducted a literature-backed analysis to place our approach in context, drawing on prior works in NLP for cybersecurity and highlighting how our solution builds upon and extends those ideas, for example, integrating knowledge from CTI extraction and the hyper-automation SOAR concept with LLMs.

## 9.3. Limitations and future work

We openly discussed current limitations, such as the need for ongoing model maintenance, explainability, and managing unseen attack patterns. The cybersecurity landscape is constantly evolving, and a static model will not be sufficient forever. However, our future directions outline a plan for addressing these issues through multi-modal analysis, continual learning, larger models, and improved human-AI collaboration. We see particular potential in integrating explainable AI to justify automated decisions and utilizing even more advanced models (such as GPT-4 or domain-specific LLMs) as security aids. We also anticipate AI

playing a bigger role in prediction and prevention, shifting IR automation from reactive to proactive approaches.

Additionally, while the proposed framework demonstrates strong performance in controlled environments, its generalization to highly dynamic and adversarial settings remains an open area for further investigation.

To guide future advancements in AI-driven cybersecurity, it is essential to identify current gaps and research opportunities. The following diagram outlines prospective areas of exploration that could enhance the robustness and adaptability of transformer-based solutions for incident response.

Taken together, these results establish transformer-based incident response not as an experimental enhancement, but as a viable operational paradigm for modern cybersecurity systems.

## Conflicts of Interest

The author declares no conflicts of interest related to the content or publication of this work.

## Data Availability Statement

This study used a combination of public research datasets and anonymized security data processed for research purposes. Public sources are cited in the manuscript. Additional data may not be publicly shared due to confidentiality constraints.

## Acknowledgments

Writing and language polishing were assisted by digital editing tools. The author reviewed and validated all content, references, interpretations, and conclusions.

## Funding

This research received no external funding. All work, including conceptual design, implementation, evaluation, and publication preparation, was funded by the author and his incorporated business entities. Zinnia Holdings and its affiliated subsidiaries internally allocated resources while committing to advancing AI and cybersecurity research. Although they did not involve grants or institutional sponsorships, the team continues efforts to secure future support and partnerships. The self-funding structure underscores the independence and originality of the research.

## Copyright and Licensing

Copyright © Paulo H. Leocadio. This manuscript is provided as a preprint for scholarly dissemination. This work is distributed under the Creative Commons Attribution 4.0 International License.



## Acronyms

- AI** Artificial Intelligence. 1–8, 12, 19–22, 24–28
- APT** Advanced Persistent Threat. 6, 18, 27
- BERT** Bidirectional Encoder Representations from Transformers. 1, 3–10, 22, 25
- CERT/CC** Computer Emergency Response Team Coordination Center. 9
- CTI** Cyber Threat Intelligence. 5, 25, 28
- CVE** Common Vulnerabilities and Exposures. 9, 27
- DARPA** Defense Advanced Research Projects Agency. 7
- EDR** Endpoint Detection and Response. 11, 16, 17, 19, 21, 28
- GAN** generative adversarial network. 23
- GNN** Graph Neural Networks. 25
- GPT** Generative pre-Trained Transformer. 1, 9
- HFD** Hugging Face Diffusers. 4, 11
- ICS** Industrial Control System. 21
- IDS** Intrusion Detection System. 5, 10, 24
- IOC** Indicator of Compromise. 8, 9, 11, 13, 17, 20
- IR** incident response. 1, 2, 4, 6, 17, 19, 27, 28
- IVAM** Investigation, Validation, Active Monitoring. 7
- KPI** Key Performance Indicator. 12
- LIME** Local Interpretable Model-agnostic Explanation. 5, 20, 25
- LLM** Large Language Model. 5–7, 9, 23, 25, 26, 28
- LSTM** Long Short-Term Memory. 6
- MTTD** Mean Time to Detect. 8, 12
- MTTR** Mean Time to Repair/Respond/Remediate. 8, 12
- NER** Named Entity Recognition. 5
- NLP** Natural Language Processing. 1–8, 10, 11, 19, 20, 23–26, 28
- OT** Operational Technology. 21
- SHAP** Shapley Additive exPlanation. 25
- SIEM** Security Information and Event Management. 2, 6, 9, 10, 16, 17, 21, 26–28
- SOAR** Security Orchestration, Automation, and Response. 2, 7, 26, 28
- SOC** Security Operations Center. 1, 6–9, 12, 17, 26–28
- TTP** tactics, methods and procedure. 25

List of Figures

1    *Integration of AI-based language models into an automated incident response system.* . . . . . 3

2    *The integration of Hugging Face Diffusers into cybersecurity workflows.* . . . . . 6

3    *Overview of evaluation metrics used across tasks and their relevance in cybersecurity workflows.* . . . . . 8

4    *Key metrics performance comparison after deploying HFD on a phishing detection dataset.* . . . . . 11

5    *An alternate view comparing metrics performance after deploying HFD on phishing detection.* . . . . . 12

6    *Time saved, accuracy, and false positive rate comparison between rule-based and transformer-driven systems.* . . 13

7    *Loading a pre-trained model, tokenizing cybersecurity data, and prep. for fine-tuning.* . . . . . 16

8    *Classification Metrics Breakdown.* . . . . . 17

9    *Precision-recall relationship during HFD fine-tuning on phishing data.* . . . . . 18

10   *HFD performance across threat types.* . . . . . 20

11   *Modular AI Agent.* . . . . . 23

12   *Synthetic Threat Simulation* . . . . . 23

13   *Cross model threat detection.* . . . . . 24

## References

- [1] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, *Computer Security Incident Handling Guide Recommendations of the National Institute of Standards and Technology*. National Institute of Standards and Technology, Aug. 2012.
- [2] Gartner, *Lack of Talent or Human Failure Will Be Responsible for Over Half of Cyber Incidents*. Gartner Research, 2022.
- [3] M. Arazzi, D. R. Arikkat, S. Nicolazzo, A. Nocer, R. Rehiman K.A., V. P., and M. Conti, "Nlp-based techniques for cyber threat intelligence," *Computer Science Review*, vol. 58, p. 100765, Nov. 2025.
- [4] D. Demirol, R. Das, and D. Hanbay, "A novel approach for cyber threat analysis systems using bert model from cyber threat intelligence data," *Symmetry*, vol. 17, p. 587, Apr. 2025.
- [5] A. Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998, 6008, 2017.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanov, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv e-prints*, Oct. 2018.
- [7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv e-prints*, Oct. 2019.
- [8] K. Mehreen, *An Overview of Hugging Face Diffusers*. KDnuggets, May 2024.
- [9] M. A. Uddin and I. H. Sarker, "An explainable transformer-based model for phishing email detection: A large language model approach," *arXiv e-prints*, 2024.
- [10] B. Ismail, S. Abdul, S. Khan, S. Sattar, and S. Muhammad, "Ai for cyber security: Automated incident response systems," *International Journal of Cyber Management*, vol. 2, no. 1, pp. 580–593, 2023.
- [11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. teusz Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877, 1901, 2020. Available at:.
- [12] M. Bromiley, *Empowering Incident Response via Automation*. SANS Institute Whitepaper, 2019.
- [13] H. Karlzen and T. Sommestad, "Automatic incident response solutions: a review of proposed solutions input and output," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, ARES 2023, pp. 1–9, ACM, Aug. 2023.
- [14] S. Sharm and P. Chen, "Deep learning applications in cybersecurity," *IEEE Security and Privacy*, vol. 18, no. 4, pp. 14–22, 2020.
- [15] M. Brundage, S. Avin, and J. Clark, *The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation*. Future of Humanity Institute, 2018.
- [16] P. Prajapati, *Hugging Face Transformers vs. Hugging Face Diffusers: What is the Buzz All About?* Elightwalk Tech Blog, May 2025.
- [17] O. Ogundairo and P. Brooklyn, "Natural language processing for cybersecurity incident analysis natural language processing for cybersecurity incident analysis," *EasyChair Preprint*, 8 2024.
- [18] M. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?": Explaining the predictions of any classifier," in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, pp. 1135, 1144, Association for Computational Linguistics, 2016.
- [19] S. Salloum, T. Gaber, S. Vader, and K. Shaalan, "Phishing email detection using natural language processing techniques: A literature survey," *Procedia Computer Science*, vol. 189, pp. 19–28, 2021.
- [20] Y. Lee, J. Kim, and P. Kang, "Lanobert: System log anomaly detection based on bert masked language model," *Applied Soft Computing*, vol. 146, p. 110689, Oct. 2023.

- [21] H. Kheddar, "Transformers and large language models for efficient intrusion detection systems: A comprehensive survey," *Information Fusion*, vol. 124, p. 103347, Dec. 2025.
- [22] I. Security, *IBM Delivers Watson for Cyber Security to Power Cognitive Security Operations Centers*. IBM Newsroom Press Release, 2 2017.
- [23] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer, "Securebert: A domain-specific language model for cybersecurity," 2022.
- [24] I. Ismail, R. Kurni, Z. A. Brat, G. A. Nelistiani, S. Heo, H. Kim, and H. Kim, "Toward robust security orchestration and automated response in security operations centers with a hyper-automation approach using agentic ai," *Information*, vol. 16, p. 365, Feb. 2025.
- [25] D. A. R. P. A. (DARPA), "Cgc: Cyber grand challenge," 2016.
- [26] P. Institute and I. Security, "Cost of a data breach report 2024," tech. rep., Ponemon Institute and IBM Security, 7 2024.
- [27] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A bayesian approach to filtering junk e-mail," in *Learning for Text Categorization*, 1998.
- [28] P. Narayanan and J. W. Carter, "Demystifying ai evaluation metrics for security-critical systems," *Journal of Information Security Research*, vol. 14, no. 2, pp. 88–104, 2023.
- [29] R. Verma and R. Shah, "Evaluating cybersecurity classifiers: The case for precision, recall, and f-measure," *IEEE Security & Privacy*, vol. 19, no. 2, pp. 84–88, 2021.
- [30] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *IEEE Symposium on Security and Privacy*, pp. 305–316, 2010.
- [31] D. Berman and N. Zilberman, "A survey of deep learning methods for cyber security," *ACM Computing Surveys*, vol. 52, no. 2, pp. 1–36, 2019.
- [32] W. contributors, "Bert (language model)," 7 2025.
- [33] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv pre print*, 7 2019.
- [34] C. Owen-Jackson, *Navigating the Ethics of AI in Cybersecurity: Balancing Privacy and Safety*. IBM Think Blog, Oct. 2024.
- [35] U. B. B. R. Ahmed Awad, "Enhancing the incident response capabilities of the uae critical infrastructure against phishing cyber security threat: An ai-based approach," in *BUID Doctoral Research Conference* (K. A. Marri, A. Abubakar, A. Awad, F. A. Mir, and K. A. Marri, eds.), Springer Nature Switzerland, 2024.
- [36] V. Barbara, M. Guarascio, N. Leone, G. Manco, A. Quarta, F. Ricca, and E. Ritacco, "Neuro-symbolic ai for compliance checking of electrical control panels," *arXiv pre-print*, 5 2023.
- [37] P. Bryant and F. NoÃ©, "Structure prediction of alternative protein conformations," *Nature Communications*, vol. 15, 12 2024.
- [38] M. Satyanarayanan, P. Bahl, R. CÃ¡ceres, and N. Davies, "The case for vm-based cloudlets in mobile computing," tech. rep., Arxiv, 10 2009.
- [39] T. Tambe, C. Hooper, L. Pentecost, T. Jia, E. Y. Yang, M. Donato, V. Sanh, P. N. Whatmough, A. M. Rush, D. Brooks, and G. Y. Wei, "Edgebert: Sentence-level energy optimizations for latency-aware multi-task nlp inference," in *Proceedings of the Annual International Symposium on Microarchitecture, MICRO*, pp. 830–844, IEEE Computer Society, 10 2021.
- [40] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. AgÃªraAg, and A. Arcas, "A supplemental figures and tables," tech. rep., ArXiv, 2017.
- [41] Y. Zhao, J. Yang, W. Wang, H. Yang, and D. Niyato, "Trandrl: A transformer-driven deep reinforcement learning enabled prescriptive maintenance framework," *IEEE Internet of Things Journal*, vol. 11, pp. 35432–35444, 2024.
- [42] M. Papoutsoglou, G. Meditskos, N. Bassiliades, E. Kontopoulos, and S. Vrochidis, "Mapping the current status of cti knowledge graphs through a bibliometric analysis," in *ACM International Conference Proceeding Series*, Association for Computing Machinery, 12 2024.

- [43] V. H. Bui, T. T. Nguyen, and H. M. Kim, “Distributed operation of wind farm for maximizing output power: A multi-agent deep reinforcement learning approach,” *IEEE Access*, vol. 8, pp. 173136–173146, 9 2020.
  - [44] Z. Hong, J. Wang, X. Qu, J. Liu, C. Zhao, and J. Xiao, “Federated learning with dynamic transformer for text to speech,” *arXiv pre print*, 7 2021.
  - [45] S. Schmitt, J. Klaehn, G. Bellec, A. Gruebl, M. Guettler, A. Hartel, S. Hartmann, D. Husmann, K. Husmann, V. Karasenko, M. Kleider, C. Koke, C. Mauch, E. Mueller, P. Mueller, J. Partzsch, M. A. Petrovici, S. Schiefer, S. Scholze, B. Vogginger, R. Legenstein, W. Maass, C. Mayr, J. Schemmel, and K. Meier, “Neuromorphic hardware in the loop: Training a deep spiking network on the brainscales wafer-scale system,” *arXiv pre print*, 3 2017.
  - [46] Y. Wang, H. Guo, G. Wang, B. Chen, and Q. Yan, “Vsmask: Defending against voice synthesis attack via real-time predictive perturbation,” in *16th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '23)*, 5 2023.
-